

## Full Length Article

## Enhanced IoT security: privacy-preserving federated learning model for accurate, real-time intrusion detection across devices

A. Puviarasu<sup>a</sup>, Sudha V K<sup>b,\*</sup><sup>a</sup> DTB-Electrical and Electronics Engineering, Caresoft Global Pvt. Ltd, Coimbatore, Tamil Nadu, India<sup>b</sup> Department of Electronics and Communication Engineering, Dr. Mahalingam College of Engineering and Technology, Pollachi, Tamil Nadu, India

## ARTICLE INFO

## Keywords:

Federated learning  
IoT networks  
Intrusion detection  
Privacy preservation  
Differential privacy  
Homomorphic encryption  
Neural networks  
Distributed learning  
Security

## ABSTRACT

Federated learning (FL) has been proposed as an effective solution in the context of intrusion detection in IoT networks, where models can be trained collaboratively with the security of raw data protection. In this paper we present a privacy-preserving FL framework based on light weight neural network, differential privacy (DP) and homomorphic encryption (HE). With a dataset of 1,191,264 instances and 47 attributes, the proposed model conducted on the IoT Intrusion Detection Dataset available on Kaggle produces overall accuracy (93.5), precision (94.2), recall (93.4), and the F1-score (94.2), with the detection time of 90–130 ms and no distinction between the attacks, where detection latency was considered in this study. At the attack level the model delivered 94.1 %, 92.5 %, and 93.6 % accuracies on DoS, DDoS, and Mirai respectively, and above 85 % accuracy on Malware and Web-based attacks. DP experiments showed that augmenting the privacy budget parameter 0.5 to 20.0 increased the levels of accuracy by 2.6 % to 94.0 %, and decreased the computational time 150 ms to 121 ms, depicting a compromise between privacy and performance. HE experiments likewise exhibited a negligible accuracy reduction (94.1 % to 93.5 %) between no encryption to complete homomorphic encryption, but required more computation time (120 ms to 200 ms). Devices-level testing demonstrated that the model had > 91 % accuracy at the low-end (0.5 GHz CPU, 128 MB memory) and up to 94.5 % accuracy with 110 ms inference time on powerful processors, irrespective of whether or not the sensor was heterogeneous, demonstrating a viable solution to the heterogeneous IT situation. Audit mechanisms further enhanced greater compliance of 0 % to 99 % with minimal reduction in accuracy (< 0.8 %). The results show that privacy-preserving intrusion detection specifically can be performed with real-time intrusion detection, high detection gene, and privacy guarantees in resource-constrained IoT networks.

## 1. Introduction

Intrusion detection in Internet of Things (IoT) networks is crucial for securing the vast array of interconnected devices that make up IoT ecosystems [1]. With IoT networks expanding rapidly in areas like smart homes, healthcare, transportation, and industrial automation, these networks face increased security risks due to their distributed nature, limited device resources, and diverse protocols [2]. Standard security approaches fail in IoT networks since most of the IoT devices are low-energy devices and cannot accommodate complex encryption or several layers of security. IDS are therefore useful in analysing flow of traffic in computer networks, performing analysis on this traffic and eventually helping or providing real-time detection of threats that may be endangering functionality of devices or data [3,4]. The recent

developments in network systems, data security and communication technologies have transformed reliability and efficiency in various domains to considerable levels. Multi-domain aggregation and street-centric methods have been used to optimize service orchestration and vehicular routing to enhance the scalability and mesh-based message delivery in an urban network [5,6]. Remedial efforts with regards to resilient swarm tracking in the presence of malicious attacks have also been explored to warrant robust control [7]. The research on circulant networks and deduplication algorithms focuses on storage efficiency and structural optimization in the meanwhile [8–10]. The invention of new forms of radar, new intrusion detection and diffusion prediction also help make intelligent systems safer and more adaptive [11–13], and economic models deal with the incentives of Sybil attacks [14].

Data provided by IDS in IoT networks indicate challenges like

\* Corresponding author.

E-mail addresses: [a.m.puviarasu@gmail.com](mailto:a.m.puviarasu@gmail.com) (A. Puviarasu), [drvksudha@gmail.com](mailto:drvksudha@gmail.com) (S. V K).<https://doi.org/10.1016/j.asej.2025.103866>

Received 29 June 2025; Received in revised form 27 August 2025; Accepted 5 November 2025

Available online 21 November 2025

2090-4479/© 2025 The Authors. Published by Elsevier B.V. on behalf of Faculty of Engineering, Ain Shams University. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

## Benefits of Intrusion Detection in IoT Networks



Fig. 1. Benefits of Intrusion Detection in IoT Networks.

distinguishing true negative between legal activity and perspective threat in low resources where both looks alike [15]. Towards this end, intrusion detection in IoT often has used more sophisticated algorithms such as machine learning and behavioural analysis to consider normal traffic and identify anomalies such as malware injection, unauthorized access, or denial of service (DoS) attacks [16]. Compared to rule-based IDS, machine learning based IDS has the advantage that the system can learn from experience and become updated with the constantly developing threat landscape on the IoT networks; it is also adaptable for each different environment such as home or industrial IoT networks. Thus, intrusion detection in IoT forms the layer of defence that will help to counteract the constant growth of cyber threats and secure users and applications in the IoT networks [17,18]. Recent developments in areas as diverse as IoT, cybersecurity, intelligent transport, and even federated learning identify the increasing demand to create resilient and efficient designs of systems. A DRL-based approach to energy-aware optimization of IoT sensors could be used to reduce energy use when gathering this data [19], and in autonomous driving, spatiotemporal cognitive risk modeling is proposed to improve predictive safety [20]. With regard to network security, new schemes involve mitigation of low-rate DoS attacks in SDN [21] and certificateless searchable For IoMT structures with LLM incorporation [22]. Federated computing systems are more robust to Byzantine actions [23] with zero-trust dynamic security computing systems [24]. Moreover, the noncooperative swarm tracking guarantees the coordination in spite of noncommercial nodes [25].

Limitations of intrusion detection for IoT networks arise from characteristics of IoT devices and environment in the following ways. One of the major issues with several IoT devices is that they have limited processing ability, memory, and computational energy for effective execution of intrusion detection algorithms [26]. This results in compromises of detection precision because simple models may not be able to capture complex attacks. Furthermore, IoT networks contain a wide range of devices that have varying configuration and dissimilarities in hardware, software and communication and hence make it difficult to design a generalized IDS that can efficiently secure the devices [27–28]. Another issue is network scalability, the greater the number of devices within the network, more data is generated that needs to be monitored, which can be both time consuming and overwhelming when it comes to IDS. In addition, IoT environments are considerably mobile compartments where devices are constantly adding or removing to/from a particular network; this poses a big problem to traditional IDS, in that, they may

not have the ability to adapt to such dynamic environments quick enough [29]. Finally, because of the inherent IoT characteristic of decentralization and distribution, the deployment and management of IDSs for IoT could be time-consuming and expensive to accomplish, challenging the IDS security standards throughout the IoT network [30–31]. The development of anomaly detection, privacy preservation system, authentication, and secure system design has been achieved in various different areas in the modern field. Industrial applications of anomaly detection implement self-supervised attention-based models and are performing better and with higher accuracy [32], and age-dependent differentially private mechanisms [33] balance data utility and protection. Keyless entry systems can be fortified with innovative forms of authentication, including a vibration-based two-factor authentication protocol [34]. Security research also has examined trade-offs of code estimation attacks [35] and the risk of covert channels through memory-disk synchronization [36]. The efficiency of hardware and privacy in the network between a satellite and the ground due to a digital twin implementation are reinforced by means of accelerating polynomial multiplications [37] and hardening the privacy in the satellite-ground digital twin network [38]. In the last, hybrid deep learning is better with intrusion detection in wireless sensor networks [39]. Fig. 1 shows the benefits of intrusion detection in IoT networks.

Deep learning for intrusion detection in IoT networks refers to the training and use of models in detecting activity anomaly or unauthorized activity in miscellaneous IoT devices. First, data is obtained from network traffic such as packet features, communication patterns and device behaviour [40–41]. This data is pre-processed and labelled to enable the deep learning regarding normal and suspicious pattern. Every once in a while, there is a need to detect spatial patterns to be captured using CNNs while the temporal patterns are captured using RNNs; this makes it easy to detect related sophisticated and evolving threats like DDoS or unauthorized access attempts. However, these deep learning models are then used within the network or at the edge on nodes that have enough processing power. They analyse streaming data in real time and raise an alarm every time a stream of activity differs from normal behaviour [42–43]. This approach allows dynamic detection because deep learning models can work with big data and regularly improve the accuracy of the detection. However, new data can be fed into the models to adapt to the unseen dangers hence making the system robust with regard to varying strategies of cyber-attacks. Using deep learning makes the IoT network safer, has a higher level of automation when identifying

intrusions, and is more efficient at recognizing threats in real-time with less dependency on human input [44].

Security, anomaly detection, and enhancements to privacy have recently focused on using graph learning, deep learning, and ensembles approaches to make these types of strategies more resilient. Models of fraud detection increasingly separate homophily and heterophily on multi-relation graphs [45], and Wi-Fi-based defenses reduce the risks of human tracking and POS password leaks [46,47]. Heterogeneous (i.e., ensemble approaches such as Func-Bagging increase the efficiency of anomaly detection [48], and graph neural networks facilitate the tracking of phishing scams within Ethereum [49]. Memory-augmented transformers help with cloud anomaly detection [50], and malware detection methods are still developing with deep learning [51]. In the meantime, new ways of dealing with DDoS attacks in SDN [52], the detecting of concurrency vulnerability [53], and the smart contract vulnerabilities [54] are offered.

Despite the fact deep learning enables intrusion detection on the IoT network, many advantages are associated with it. There also exist numerous disadvantages, which might impede the process. The downside of this, however, is associated with the requirement to have volumes of training data which may be challenging to obtain in IoT applications particularly when involving Mikro and variable networks. The inadequacy or irrelevancy of data leads to poor model performance on many failure cases and high false positive/negative detection [55]. Moreover, deep learning is computationally demanding requiring high processing power and memory, most of which is beyond the capacity of many IoT devices that may have limited resources. One consequence of this restriction is the tendency to depend on edge devices/central servers which may cause single points of failure or latency in detection [56-57]. Finally, such models may also be vulnerable to an adversarial attack, with an adversarial party causing input data to mislead the model, compromising the usefulness of the intrusion detection mechanism. These issues together demonstrate the importance of a cautious step-by-step deep learning solution applicability and monitoring of IoT intrusion monitoring [58]. FL offers a novel solution that provides a way to train a global model collaboratively without sharing raw data with IoT devices. Nevertheless, the best counter to this is to keep and maintain privacy in FL, which — as strategies should be used such as differential privacy and homomorphic encryption [59]. In this work, a privacy-preserving FL framework to IoT intrusion detection is proposed with an emphasis on secure and distributed learning and robust intrusion detection performance.

In this paper, we present the new privacy-preserving Federated Learning (FL) framework of IoT intrusion detection, which helps solve both tasks to protect data privacy and balance the resource constraints in IoT landscapes. Contrary to the traditional centralized methods in which raw data is at risk of breaching security, our framework provides a distributed training methodology in which data is local to each device thus preventing data leakage. The proposed system proposes an extremely light neural network architecture, which also works best when detecting intrusion in real time using low-resource IoT devices and, therefore, become very handy in implementation. Enhanced privacy is further supported by the use of differential privacy mechanisms, which include the additions of Gaussian noise to models and using homomorphic encryption to support sharing models privately, associating privacy guarantees as well as security of sensitive updates emerged in aggregation.

### 1.1. Main contributions of the work

- **Privacy-Preserving FL Framework:** Developed a FL framework for IoT intrusion detection that enables distributed model training without requiring raw data to leave individual IoT devices, ensuring data privacy.
- **Lightweight Neural Network Architecture:** Designed a resource-efficient neural network architecture optimized for IoT devices

with limited computational and memory resources, enabling real-time intrusion detection in constrained environments.

- **Differential Privacy Implementation:** Integrated differential privacy through Gaussian noise addition to model updates, protecting individual device data and providing a mathematical privacy guarantee while allowing collaborative learning.
- **Homomorphic Encryption for Secure Model Sharing:** Applied homomorphic encryption to model updates before sharing them with the central server, allowing secure aggregation and preventing exposure of sensitive information in the model updates.
- **Audit Mechanisms for Privacy Compliance:** Incorporated audit mechanisms to monitor and verify the consistent application of privacy-preserving techniques, ensuring compliance with privacy standards throughout the FL process.
- **Adaptability to Diverse IoT Attack Types:** Demonstrated the model's adaptability in identifying various IoT-specific attacks by leveraging a flexible yet robust feature extraction and processing methodology.

The remainder of this paper is organized as follows: [Section 2](#) provides a review of related studies on intrusion detection techniques and privacy-preserving FL approaches, highlighting advancements and limitations in securing IoT networks. [Section 3](#) details the proposed methodology, including the model architecture, privacy-preserving techniques, and FL configuration tailored for IoT environments. [Section 4](#) describes the results and discussion, presenting the model's performance in intrusion detection and evaluating its efficiency, privacy, and adaptability. Finally, the Conclusion and Future Scope section summarizes the study's contributions and suggests areas for enhancing the framework, such as advanced encryption and scalability for diverse IoT applications.

## 2. Related works

Examining the issue of security in the IoT framework indicates that intrusion detection is becoming pertinent due to increased use of IoT devices. The big data analytics used by the machine learning algorithms assists in sorting out intrusions by isolating abnormalities in the compiled big data. Conventional centroid-based learning paradigms though involve the gathering of data to the centre where learning occurs. The strategy has privacy concerns because massive amounts of personal information are placed in the possession of a single corporation that is susceptible to attacks. In this regard, a potential solution presents itself with FL, in which several parties can be trained based on the same model without access to raw data [60]. The risk of data leakage is further reduced and the effectiveness of training by such privacy-preserving. The federation-based intrusion detection system (IDS) in which several client devices synchronize with a global model to the server. This is transmitted back to clients in an aggregated form so that further analysis may be carried out. Experiment demonstrates that federated-IDS achieves a high detection rate and optimal security / performance ratio in IoT.

This article provides a detailed perspective on approaches to network security by implementing deep learning in intrusion detection. Clearly, the issues in the real life are dissimilar due to the high dimensions of the data considered that introduces problems like the vanishing gradient in deep-learning models. Nonetheless, training on network traffic data again has the problem of concentration, where concentrating data is a privacy standpoint as the data is exchanged with third-party servers. A Residual Network (ResNet)-based model (w/ FL) can be used to solve these issues [61]. ResNet helps mitigate the vanishing gradient problem and FL allows multiple ISPs or clients to cooperatively train a model without raw data exchange. Along with this process, the efficiency of the information retrieval also improves with the added advantage of the privacy that is preserved. The given solution provides possible means of providing improved and secure intrusion detection without the infringement of data privacy in interconnected IoT systems.

In order to overcome the threats of the attackers aiming to penetrate vehicular networks in IoT edge implementations, a FL-based intrusion detection system (FL-IDS) is proposed. In this direction, the FL-IDS system helps data privacy through integration of an element of local learning whereby the model upgrade only is sent to an aggregation server that helps to integrate to a superior detection model. This architecture applies machine learning classifiers, deep learning classifier, logistic regression (LR) and convolutional neural networks (CNN) to reduce the transportation IoT-based attacks [62]. Two configurations were considered in the experiments, two clients in the testbed 1 and four in the testbed 2. Using the performance analysis, the proposed FL-IDS can achieve high accuracy rates 94 % and 99 % and negligible loss 0.28 and 0.009 further making fair comparison with the centralized approaches. This piece of work confirms that FL-IDS safeguards and enhances the security of the IoT CAVs against sophisticated cyber-attacks. The attacks on the IoT infrastructure are fast-growing because more IoT devices are used. IDSs therefore have evolved to become a central part of cybersecurity given that it assists in detection of the anomalous activities. Although the majority of the existing IDSs use ML, most of them are centralized, that is, the information that the devices use are transmitted to a central server, and this compromises privacy. FL suggested as privacy preserving solution to the centralized ML in intrusion detection of IoT systems [63]. FL is validated on real data using IoT devices known as ToN\_IoT and a shallow artificial neural network and the federated averaging (FedAvg) aggregation method, to classify smart environments using binary and multiclass classifications. FL is personal, in that, devices do not train on the server. In this way, the study demonstrates that FL makes it possible to achieve the combination of a fast IoT intrusion detection solution and privacy preserving with, in addition, increased accuracy and precision and, more importantly, a significant boost in the recall. Growth of IIoT to support friendly human machine interface has also created great amount of heterogeneous data and consequently a great number of exposure to hack attacks. This data should be preserved above all, and ML algorithms are frequently used to analyse network traffic, such as to detect an intrusion. Classical ML models however demand centralized dataset and hence cannot be implemented as secure intrusion frameworks. In this regard, we apply an FL-based model to IIoT scenarios in which fluctuations are shared with a central server with centralized data [64]. GRUs re-enter the scene to discern the temporal patterns whereas DRL is explored to eliminate the trash of the good and quality IIoT devices in a manner than enhances the model aggregation rate and energy as well. The method does not entail any independence and identical distribution of the data as is the case with other methods that were previously used. The analysis of recent IIoT datasets proves that the FL-based framework performs better than FL and similar non-FL methods in the metrics of accuracy, precision, recall, and F1-score, and the proposed method is safe to use to detect an intrusion in IIoT.

### 3. Methodology

The suggested approach to intrusion detection privacy-preserving an IoT network involves a FL framework containing lightweight neural networks to run on IoT devices. The local models are trained by each device on its data and learn the pattern of intrusions without exchanging them with a central server. Differential privacy was used on every model update, with Gaussian noise added in order to protect individual data points but still keep useful learning signal. Homomorphic encryption is used to perform model updates remotely, and the central server could aggregate the updates securely without operating on unencrypted data. Privacy is tracked by an audit mechanism that records parameters like noise level, privacy level and encryption status to determine a privacy compliance. The updated global model is propagated at the end of aggregation to the central server, which is then distributed back to each device until the model converges. Such an approach would guarantee the efficient, safe intrusion detection and the preservation of the privacy

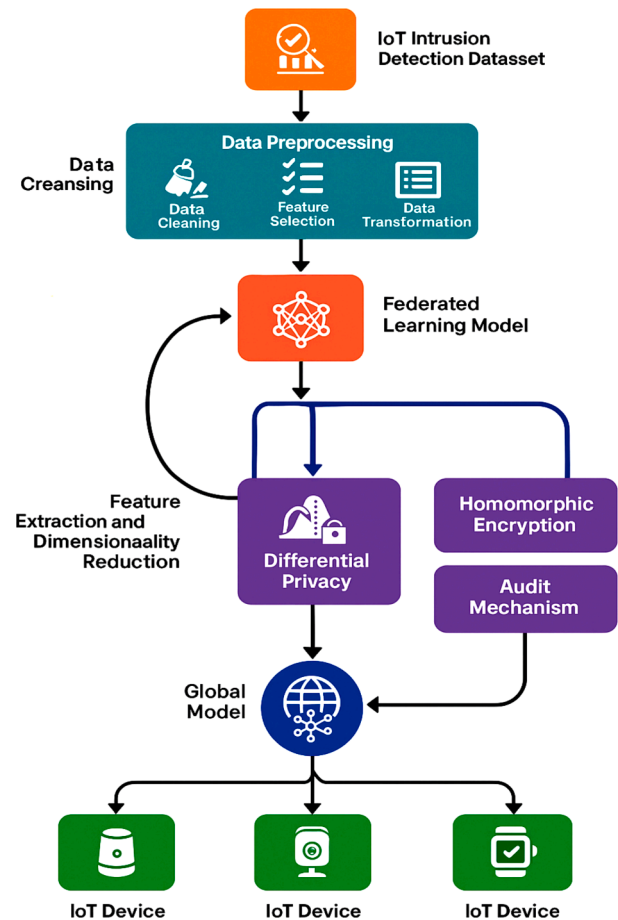


Fig. 2. Architecture of the Proposed Privacy-Preserving Federated Learning Framework.

of the IoT data. Fig. 2 shows the architecture of the proposed privacy-preserving federated learning framework for IoT intrusion detection.

#### 3.1. IoT intrusion detection dataset

IoT Intrusion Detection Dataset published on Kaggle serves as a large repository of data to design IDS and test them on IoT networks [65]. Such a dataset proves particularly valuable when used by researchers and developers interested in investigating IoT security because it presents quite a significant number of labelled data that have been replicated by various forms of network intrusion. These are Distributed Denial of Service (DDoS), Brute Force, Spoofing, Denial of Service (DoS), Reconnaissance (Recon), Web-based attacks and Mirai malware attacks among others. All of them are further divided into categories that give us an idea of the specific types of intrusions and the type of directions that is most common to IoT networks. With 1,191,264 network data pieces and 47 various pieces of features, such an amount of data is reasonably diverse and rich enough to train machine learning algorithms. The features solve a wide range of network features to facilitate models to differentiate between benign and malicious activities. These aspects are indispensable in formation of the model that is applied in prediction of various intrusions with high confidence. This provides detailed data on attacks so that complex IDS systems can be designed to distinguish between various forms of intrusion. In this regard, training and testing the models in this degree of detail is valuable in the attainment of the intended robustness and flexibility to attacks in the real world IoT.

### 3.2. Data preprocessing

#### 3.2.1. Data cleaning

The initial stage of preprocessing is usually data cleaning which is aimed at washing out noise of the data that will later consequently impair its proper processing. Intrusion detection data have the tendency to include missing values, which when not addressed lead to misleading information. Such imputation methodology as Median imputation simply replaces missing values with the median value of the given feature, without skewing the nature of the whole data set with such addition of radically different value. The same approach is also used with intrusion datasets because median can more effectively capture how outliers may interfere with normal traffic versus the later. Also, in case of duplication of an individual in the records, then it must be removed since it will be much data on those records bringing bias on the data. Frequent data cleaning constitutes the foundation on how such a model would forecast successfully without any hassle associated with faulty data.

Let  $X = \{x_1, x_2, \dots, x_n\}$  be a feature with missing values, for any missing value  $x_j$  in  $X$ , input as follows:

$$x_j = \text{median}(X) \quad (1)$$

Where  $\text{median}(X)$  represents the median of non-missing values in  $X$ .

**Duplicate Removal:** For any instance  $X_i$  that appears more than once, remove all but one instance:

$$X' = X - \{X_i | X_i \text{ is a duplicate}\} \quad (2)$$

#### 3.2.2. Data transformation

Data transformation makes the dataset better suited to consumption by Machine Learning models. As case in categorical variables e.g., attack type categories observed in the analysis that was performed in the background, we use one hot encoding to convert them to binary inputs that are understood by the algorithms. To illustrate, categories are transformed to columns of binary values that indicate presence or absence of a certain type of attack. Furthermore, chronologic parameters, such as packet size, or connection length are scaled, to the result that every value becomes now of comparable measurement. This is relevant to distance-based algorithm and neural networks since they are sensitive to differences in the magnitude of featured items. Normalization enhances convergence rate of the training process and achieve greater accuracy enabling models to properly distinguish between normal and malicious operations within the network.

**One-Hot Encoding:** For a categorical variable  $C$  with  $k$  category  $c_i$  is transformed into a binary feature:

$$\text{one}_{\text{hot}}(c_i) = \begin{cases} 1 & \text{if } C = c_i \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

**Normalization:** For each numerical feature  $X = \{x_1, x_2, \dots, x_n\}$ , normalization scales values to a range, e.g.,  $[0,1]$ :

$$x'_i = \frac{x_i - \min(X)}{\max(X) - \min(X)} \quad (4)$$

Alternatively, Z-score normalization standardizes the data to have a mean of 0 and a standard deviation of 1:

$$x'_i = \frac{x_i - \mu}{\sigma} \quad (5)$$

Where  $\mu$  is the mean of  $X$  and  $\sigma$  is the standard deviation.

#### 3.2.3. Feature selection

The importance of feature selection is not only limited to reduce the dimensionality of data and maximize attribute discovery that is pertinent to intrusion detection. Using simple techniques, like correlation analysis, one can choose the features with the highest information density regarding intrusions and discard those that are either noisy or

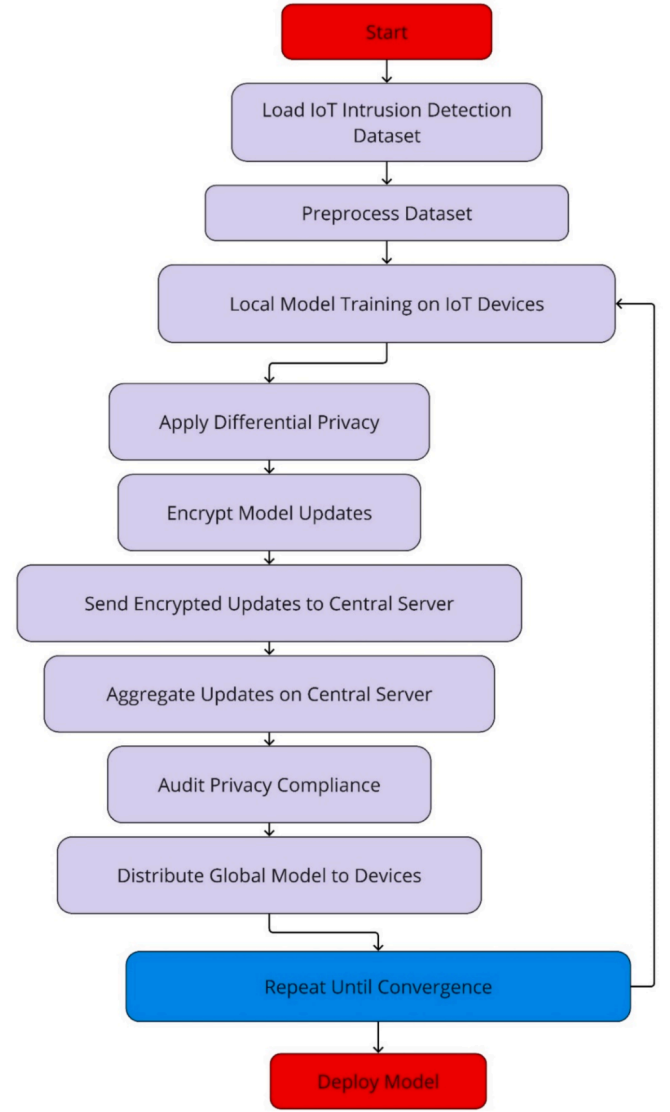


Fig. 3. Flowchart of Proposed Model.

redundant. An example, where we have features that are closely related, one of the features can signify the rest. Fig. 3 shows the flowchart of proposed model.

The resulting reduction does not only provide optimal performance to the model, but also enhances readability because the model makes projections based on the most important parameters to network traffic behavior. The feature selection contributes to the reduction of the time needed to train the model since one deals with the most relevant data, therefore, reducing the time of computations when performing the prediction.

**Correlation Analysis:** Given two features  $X$  and  $Y$ , the correlation  $\rho_{XY}$  is calculated as:

$$\rho_{XY} = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y} \quad (6)$$

Where  $\text{cov}(X, Y)$  is the covariance of  $X$  and  $Y$  and  $\sigma_X$  and  $\sigma_Y$  are their standard deviations. Features with  $|\rho|$  near 1 are highly correlated and are the candidates for removal.

#### 3.2.4. Outlier detection

A model can be biased by anomalies or observations that differ strongly with the rest of the observations, and can reduce performance.

In the case of inspecting the intrusion detection concepts, there is a likelihood that outliers are applied to explain situations that occur beyond the network trends. Such clustering methods like DBSCAN (Density-Based Spatial Clustering of Applications with Noise) can identify these kinds of records because the method calculates data density. DBSCAN may also apply to the IoT network data because it enables the detection of traffic cluster, which would otherwise be an outlier, as opposed to close neighborhood technique. The removal of such abnormal observations will prepare the data in a manner such that the data is no longer noise contaminated and professionals can train the models that recognize the regular causes of flow intrusion.

In DBSCAN, a point  $p$  is considered an outlier if it has fewer than  $\text{minPts}$  neighbors within distance  $\epsilon$ :

$$\text{outlier}(p) = \begin{cases} \text{True} & \text{if } |\{q \in X : d(p, q) \leq \epsilon\}| < \text{minPts} \\ \text{False} & \text{otherwise} \end{cases} \quad (7)$$

Where  $d(p, q)$  is the distance function between points  $p$  and  $q$ .

### 3.2.5. Data balancing

During the development of the model, data balancing is primary, particularly when the datasets are comprised of regular and rare intrusions. Skewed data imply that certain classes strongly override others excessively, and when learning occurs some models will overfit the majority classes at the expense of the minority classes. The methods that are applied to handle this include over sampling of the minority class or undersampling the majority class with techniques like SMOTE (Synthetic Minority Over-sampling Technique). The concept of SMOTE is to generate new examples of the minor class and, thereby, enhance its coverage without decreasing the copy of the examples. Equally cutting the data to imply less possibility of overweighting rare attacks during training the model to create a balanced level of sensitivity to each intrusion type is logical.

For each sample  $x$  in the minority class, a synthetic sample  $x_{\text{new}}$  is created by interpolating with one of its  $k$ -nearest neighbors  $x_{\text{nn}}$ :

$$x_{\text{new}} = x + \lambda \bullet (x_{\text{nn}} - x) \quad (8)$$

Where  $\lambda \sim U(0, 1)$  is a random scalar drawn from a uniform distribution.

### 3.2.6. Data splitting

The split of the dataset into the training and the validation set and the test set is an extremely important procedure that enables one to estimate and adjust the anticipated model. There is a common data further split 70–15–15, which enables to assign an appropriate quantity of training data, and the remaining percentage is divided into validation and test sets to evaluate the independent model. The training dataset is applied in the model building, and validation dataset is applied either in the selection of the correct parameters and the avoidance of over training cases. Finally, there is the test set which is utilized in finding out the real performance of the model in the real-life situation. This segregation avoids the problem of overfitting to the training data, which is of utmost importance to any IDS as this needs to be correct in its reaction to previously unseen forms of attacks.

### 3.3. Feature extraction and dimensionality reduction

It has been identified that feature extraction plays a significant part of constructing an effective IDS on IoTs. The subsequent step would be selecting and retrieving features related to network intrusions which enhance the model's ability to detect malicious activities. Common parameters often used, to base the development of intrusion-detection system on, in the case of IoT include size of packets, connection time, protocols used, among other network parameters. Such information, e.g. the size of the packet, can indicate an illegitimacy data transfer as such is most of the time an indicator of a Distributed Denial of Service (DDoS) attack. The subsets where network activity aspects have significant

results are readily identified, facilitating isolation of models to identify what is normal and anomalous traffic to give more precision and eliminate insignificant traits.

Let  $X$  represent the dataset with  $n$  instances and  $m$  original features:

$$X = \{x_1, x_2, \dots, x_n\}, x_i = \{x_{i1}, x_{i2}, \dots, x_{im}\} \quad (9)$$

To extract relevant features, select a subset  $F \subseteq \{1, 2, \dots, m\}$  where each feature  $f \in F$  has high relevance for detecting network intrusions (e.g., packet size, connection duration, protocol type). The transformed dataset  $X'$  containing only these selected features is:

$$X' = \{x'_1, x'_2, \dots, x'_n\}, x'_i = \{x_{if} : f \in F\} \quad (10)$$

Feature selection and reduction is the second most significant step after feature extraction which is essential due to limited processing abilities of IoT devices. It is also typical that the intrusion detection datasets have very large numbers of features in them and the majority if not all of the features may not be needed in making classification of the attacks. Such a complex structure of the dataset needs to undergo dimensionality reduction so that to this end, in this paper, t-SNE (t-distributed Stochastic Neighbor Embedding) is utilized to plot down the high dimensional data into a low dimensional structure such that similar data points are in proximity to each other and vice versa. This enables finding groups and structures in regions that can appear arbitrarily scattered in greater dimensions and, consequently, assists in finding outliers. Nonetheless, it can be an advantage as well in that smaller dimension means a higher speed of computation and reduces the memory requirements, making it suitable in online intrusion detection in IoT device.

To reduce the dimensionality of  $X'$  using t-SNE, let  $X'$  be a high-dimensional dataset with  $d$  features ( $d = |F|$ ) that we aim to project into a lower-dimensional space (e.g., 2D or 3D). The probability that points  $x_i$  and  $x_j$  are space is:

$$p_{ij} = \frac{\exp(-\|x'_i - x'_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x'_i - x'_k\|^2 / 2\sigma_k^2)} \quad (11)$$

Where  $\sigma_i$  is the variance of the Gaussian distribution centered at  $x'_i$ . In the low dimensional space (2D for simplicity), the similarity between the mapped points  $y_i$  and  $y_j$  is calculated as:

$$q_{ij} = \frac{\left(1 + \|y_i - y_j\|^2\right)^{-1}}{\sum_{k \neq i} \left(1 + \|y_i - y_k\|^2\right)^{-1}} \quad (12)$$

t-SNE minimizes the Kullback-Leibler divergence between the two distributions  $P = \{p_{ij}\}$  and  $Q = \{q_{ij}\}$  to achieve the mapping:

$$KL(P||Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (13)$$

This process also involves assigning relevancy score to each feature. Therefore, some of features can be easily deleted because they are redundant or irrelevant on the basis of feature importance. Quantitative environments such as correlation matrix or feature importance measures of machine learning systems such as a random forest allow ranking of features according to the extent of assistance they provide the solution to the intrusion detection problem. Estimating unnecessary features reduces the computational load as well as over-learning and enhances the capacity to generalize between various kinds of attacks. Correlation or feature importance scores may be used in order to detect the degree of relevance.

Feature importance from a model like Random Forest can be calculated. For a feature  $X_f$ , its importance score  $I_f$  can be defined as the average reduction in the impurity (e.g., Gini impurity or entropy) across all trees in the forest where  $X_f$  is used:

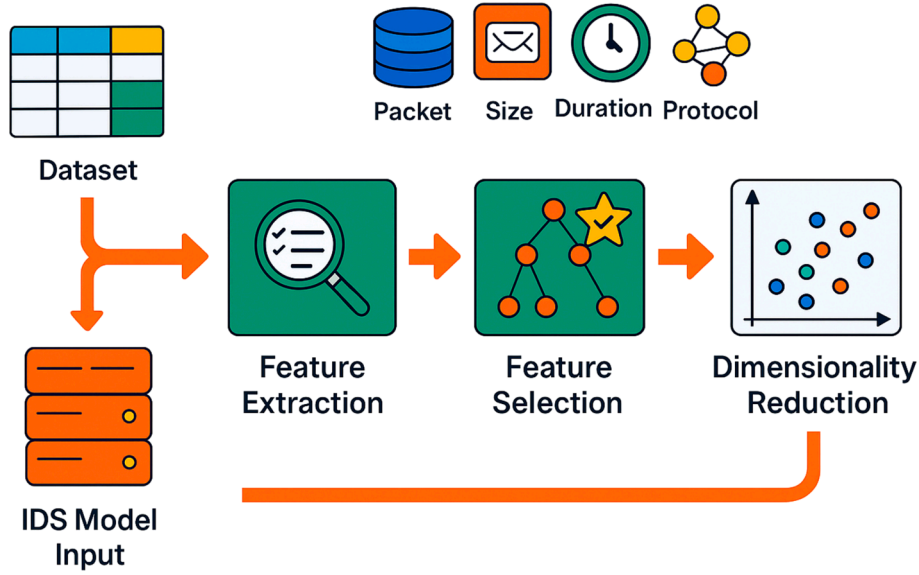


Fig. 4. Architecture of Feature Extraction and Dimensionality Reduction.

$$I_f = \frac{1}{T} \sum_{t=1}^T \Delta \text{Impurity}(X_f, t) \quad (14)$$

Where  $T$  is the total number of trees and  $\Delta \text{Impurity}(X_f, t)$  represents the reduction in impurity when splitting on  $X_f$  in tree  $t$ . After calculating relevance scores, discard features with relevance scores below a threshold  $\theta$ :

$$F = \{f \in F : I_f \geq \theta\} \quad (15)$$

The reduced dataset containing only relevant features  $F$  IS:

$$X^n - \{x_1^n, x_2^n, \dots, x_n^n\}, x_i^n = \{x_{if} : f \in F\} \quad (16)$$

Fig. 4 illustrates the preprocessing pipeline where dataset undergoes feature extraction, selection, and dimensionality reduction, producing optimized IDS model input for accurate intrusion detection.

### 3.4. Federated learning model

The FL approach of the IoT intrusion detection begins with the prospective modeling of the model that can be trained to fit the limitations of the IoT devices in the case of processing power, memory, and energy constraints. A small CNN or a shallow MLP is an example since a neural network of that type is neither very large nor has insufficient learning potential. They also have the capacity to process containing couples in an efficient way and distinguish normal and malicious behaviors. As the target devices are the IoT devices, the model should be restricted to have a small number of layers and parameters to emerge smoothly on these devices without a negative impact on detection performance.

Let  $f_\theta$  represent a lightweight neural network model with parameters  $\theta = \{W, b\}$ , where  $W$  represents the weights and  $b$  represents the biases in the network. For simplicity, we denote the model as:

$$f_\theta(x) = \sigma(W \bullet x + b) \quad (17)$$

Where  $x$  is the input (e.g., network features), and  $\sigma$  is an activation function (e.g., ReLU or sigmoid). The network has also been simplified to have few layers and parameters to ensure it can be run effectively on IoT devices. Learning of the models in the FL architecture occurs in the device that does not transmit its original data to the hub server. The overall strategy behind the conventional machine learning is that all the machines are typically fed into a gigantic central repository to be trained

in the model. However, such step is absent in FL or at least it is not necessary there. Rather, each IoT device processes its data and trains the model parameters allowing weights and biases, and sends them to the central server. The server will receive these updates and will perform model averaging, i.e. compute the joint model, which is useful since it captures the learning of all the devices. A major benefit of such an approach is that it does not convey raw data, thereby eliminating the problem of privacy concerns, particularly in the case of an IoT network where privacy-sensitive data might be traversing it.

In FL, each device  $i$  (where  $i = 1, 2, \dots, n$ ) has its own local dataset  $D_i$ . The central server initializes a global model with parameters  $\theta^0$  and distributes this model to all participating IoT devices. This initialization step can be represented as:

$$\theta^0 \rightarrow \{\theta_i^0\}_{i=1}^n \quad (18)$$

Where  $\theta^0$  is the initial model parameters on the server, and  $\theta_i^0$  are the copies sent to each device  $i$ . Each IoT device  $i$  trains the model locally on its own data  $D_i$  for a fixed number of epochs or until convergence. The local model parameters  $\theta_i$  after training are updated based on the local objective:

$$\theta_i = \underset{\theta}{\operatorname{argmin}} \frac{1}{|D_i|} \sum_{x \in D_i} L(f_\theta(x), y) \quad (19)$$

Where  $L$  is the loss function (e.g., cross-entropy loss for classification tasks),  $x$  are the features, and  $y$  are the corresponding labels. This step results in a set of locally optimized parameters  $\theta_i$  for each device. In FL the initial feasible step would be to install the base model on the server and distribute it to all the IoT appliances. The initialization defines the process during which all the devices start the process at random or pre-trained weights. Each of the devices is trained starting with the same initial point, and once the given model has been initialized, it is broadcast. This standard boot will assist in making a unity of the respective FL action as devices will be syncing to a selected foundation. The model converges more rapidly to an optimal solution that is essential in the IoT environment since optimization has been set at the initial stages of the process across the multiple devices. After local training, each device  $i$  sends its updated model parameters  $\theta_i$  to the central server. The server then aggregates these updates to form a new global model  $\theta^{t+1}$  at each round  $t$  of training. A common aggregation method is Federated Averaging, defined as:

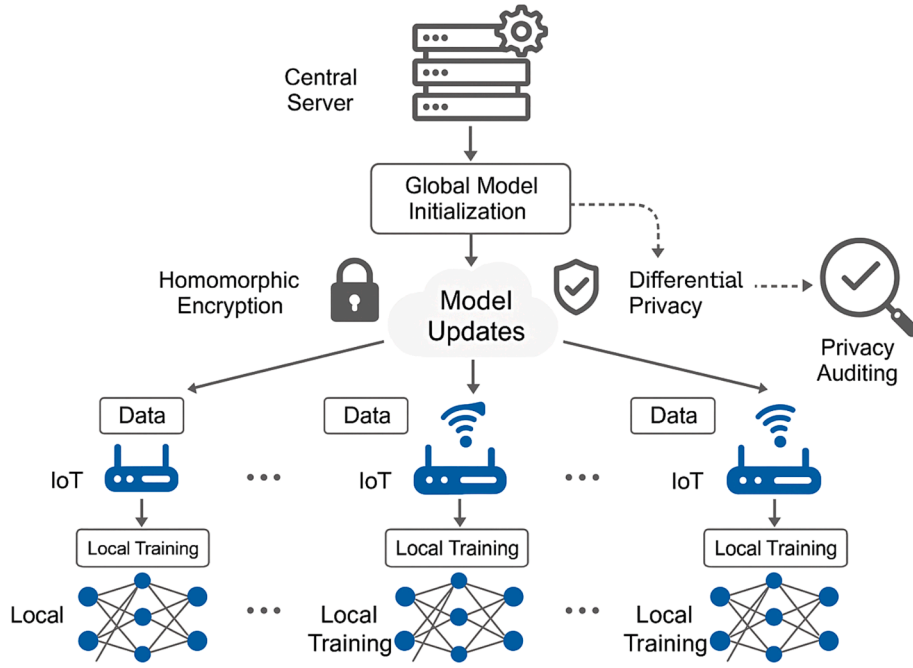


Fig. 5. Federated Learning Architecture for IoT Intrusion Detection.

$$\theta^{t+1} = \frac{1}{n} \sum_{i=1}^n \theta_i^t \quad (20)$$

Where  $\theta_i^t$  represents the model parameters from device  $i$  after round  $t$  of training, and  $n$  is the total number of devices. Once the central server computes the updated global model  $\theta^{t+1}$ , it distributes this model back to each IoT device for the next round of local training:

$$\theta^{t+1} \rightarrow \{\theta_i^{t+1}\}_{i=1}^n \quad (21)$$

In this training, which is repeated in a loop of local training, parameter aggregation, and global model update until the model converges or 10 rounds are completed, the client updates parameters and the server aggregates parameters before updating the global model. Using the FL, IoT devices can build strong intrusion detection model as a collaborative effort, with the raw data stored on the individual devices. Fig. 5 depicts the federated learning architecture where IoT devices perform local training, share encrypted updates with differential privacy, and the central server aggregates models securely.

### 3.5. Privacy-Preserving enhancements

Data privacy is however vital in the FL setting (to comply with IoT intrusion detection), mainly because in the case of IoT networks, information is usually of a very sensitive proportion. Differential privacy is one of the most practical privacy preservation methods of all. Therefore, differential privacy ensures that the contribution of a unique identifier of each data point in the dataset of the IoT device remains intact in the model updates that is sent to the central server. This is achieved by adding controlled noise into the updates of every device before transmission in a manner that masks specific data characteristics. As an example, adding noise like Gaussian with a certain variance to model weights implies that one cannot reverse discriminate individual data points, we have provably good privacy. The tradeoff between model accuracy and privacy is enabled by applying differential privacy to each IoT device, where an update that does not leak that privacy-sensitive information is permitted.

In differential privacy, noise is added to the model updates to ensure individual data points cannot be inferred from the shared parameters.

For each model update  $\theta_i$  on device  $i$ , we add random noise  $N(0, \sigma^2)$  (where  $\sigma$  is the standard deviation of the noise), resulting in a noisy update  $\theta_i'$  before sending it to the central server:

$$\theta_i' = \theta_i + N(0, \sigma^2) \quad (22)$$

Where  $N(0, \sigma^2)$  is Gaussian noise with mean 0 and variance  $\sigma^2$ . This noise addition ensures that the change in any single data point minimally affects the model update, providing a level of privacy protection. The privacy loss, also known as the privacy budget  $\epsilon$ , quantifies the degree of privacy. For each update,  $\epsilon$ -differential privacy can be defined by the probability ratio:

$$\Pr[M(D) = o] \leq e^\epsilon \cdot \Pr[M(D') = o] \quad (23)$$

Where  $M$  is the mechanism (the noisy update) applied to two neighboring datasets  $D$  and  $D'$  that differ by a single data point, and  $o$  is any possible output. Smaller values of  $\epsilon$  imply stronger privacy. The homomorphic encryption can be involved in the model weight sharing process to reinforce the privacy in FL frameworks. On homomorphic encryption, the IoT devices would encrypt their upgraded model so that it could be transferred to the central server. This kind of encryption enables the server to perform operation on the data without necessarily deciphering data. Thus, the server can aggregate the encrypted model updates made in all devices and derive a global model without access to the original data. Homomorphic encryption ensures that even when the server is breached, data is not compromised. Although homomorphic encryption can be countered with increased computational burden, the study in this area has led to its feasibility in FL applications particularly in light weight neural networks that are fit to do jobs in resource constraint settings like IoT.

In homomorphic encryption, each device  $i$  encrypts its model update  $\theta_i$  before sending it to the central server. Let  $E(\theta_i)$  denote the encrypted model update using a homomorphic encryption scheme. Homomorphic encryption enables computation to be performed directly on encrypted data, so the server can aggregate encrypted updates without decrypting them. For the aggregation of  $n$  encrypted updates  $E(\theta_i)$ , the server computes:

$$E\left(\sum_{i=1}^n \theta_i\right) = \sum_{i=1}^n E(\theta_i) \quad (24)$$

This homomorphic property allows to make sure that the encrypted sum is the same as the sum of encrypted updates and thus the central server can obtain the aggregated global model without accessing individual unencrypted updates. The  $E(\sum_{i=1}^n \theta_i)$  is returned to the devices (to be decrypted and locally processed) by being transmitted by the server after aggregation. With audit procedures, data security and privacy issues can also be dealt with during implementation of the FL process. These mechanisms monitor data flow and update of models concerning evidence of any form of data leakage or data usage that might be causing a privacy issue. As an example, the noise level of the differential privacy, encryption updates, and adaptation to the laws protecting data fall under the audit framework. They also inspect whether the privacy enhancing mechanisms such as differential privacy, or encryption, are set up properly in all connected devices. By the above auditing mechanisms, the managers of IoT and other important stakeholders will be guaranteed that all privacy measures have not been breached and therefore, the FL aims at keeping the information not to be published.

An auditing measure guarantees that privacy-protection measures and the level of compliance are always observed. For example, logging the noise level  $\sigma$  and encryption status for each update allows for verifiable privacy adherence. For each device  $i$ , an audit log  $A_i$  is maintained with entries such as:

$$A_i = \{\sigma_i, \epsilon_i, \text{encryptionstatus}, \text{timestamp}\} \quad (25)$$

Where  $\sigma_i$  is the standard deviation of noise added to the model update for differential privacy,  $\epsilon_i$  is the privacy budget used, *encryptionstatus* is a Boolean indicating whether the update was encrypted, and *timestamp* is the records the time of each update. Auditing then involves verifying each log entry  $A_i$  to ensure:

$$\forall i \text{ if } \sigma_i \geq \sigma_{\min} \text{ and } \epsilon_i \leq \epsilon_{\max} \quad (26)$$

Where  $\sigma_{\min}$  and  $\epsilon_{\max}$  are pre-defined privacy thresholds that devices must adhere to. This verification ensures that each device meets the privacy-preserving requirements.

Although both differential privacy and homomorphic encryption can greatly raise the level of security in the federated learning pipeline, when they are combined, they present computational considerations that need to be addressed with special caution in the IoT applications. Because it adds noise to model updates in order to provide statistical privacy guarantees, differential privacy may have minor effects on the accuracy of classification and add complexity to latency. In the same way, homomorphic encryption can enable secure aggregation of encrypted updates, and the extra cryptographic work incurs larger inference latency and energy costs on resource-constrained devices. These trade-offs pose the necessity of a compromise between the desire to give excellent privacy guarantees and easy feasibility. To alleviate this, it considers optimised encryption schemes and privacy budgets which are analyzed quantitatively in the results section to gain empirical insights on the cost-benefit trade-offs between accuracy, latencies and privacy.

**Algorithm: Privacy-Preserving federated learning for IoT intrusion detection**

**Input:** Local dataset  $D_i$  on each device  $i$  containing features relevant for intrusion detection

Central server for model aggregation

Lightweight neural network model architecture  $f_\theta$

Privacy parameters: noise level  $\sigma$  for differential privacy, encryption keys for homomorphic encryption, and privacy budget  $\epsilon$

**Output:** Trained global model  $\theta$  with privacy-preserving mechanisms

**Model Initialization on Central Server**

$\theta^0 = \{W, b\}$  // Initialize the global model parameter

$\theta_i^0 = \theta^0 \forall i \in \{1, 2, \dots, n\}$  // Distribute initial model

(continued on next column)

(continued)

#### Local Training on IoT Devices

For each FL round  $t$ :

For each device  $i$  in parallel

$\theta_i^t = \theta^{t-1} - \eta \nabla L(f_{\theta^{t-1}}(D_i), Y_i)$  // Compute the local model parameters

$\Delta \theta_i = \theta_i^t - \theta^{t-1}$  // Compute the model update

$\Delta \theta_i = \Delta \theta_i + N(0, \sigma^2)$  // Add Gaussian noise

$E(\Delta \theta_i) = \text{Encrypt}(\Delta \theta_i)$  // Encrypt the noisy model update

$A_i = \{\sigma, \epsilon, \text{encryptionstatus}, \text{timestamp}\}$  // Record an audit log entry  
Sent  $E(\Delta \theta_i)$  to the central server.

#### Model Aggregation on Central Server

$E(\sum_{i=1}^n \Delta \theta_i) = \sum_{i=1}^n E(\Delta \theta_i)$  // Use homomorphic encryption

$A_{\theta^{t+1}} = \text{Decrypt}(E(\sum_{i=1}^n \Delta \theta_i))$

$\theta^{t+1} = \theta^t + \eta \Delta \theta^{t+1}$  // Update the global model parameters

#### Distribute Updated Model to IoT Devices

Distribute the updated global model

#### Privacy Auditing

For each device  $i$  and each federated round  $t$ :

$\sigma \geq \sigma_{\min}$  // Verify the audit log  $A_i$

$\epsilon \leq \epsilon_{\max}$

#### Termination

Repeat the steps until the model converges reaches the maximum number

Return the Final Global model  $\theta$

**End Algorithm**

### 3.6. Novelty of this work

A novelty of the designed work is that it integrates privacy-preserving tools into a FL framework specifically designed to support intrusion detection in IoT networks. In contrast to the legacy centralized intrusion detection systems where raw data should be transferred to a central server, our solution makes distributed training possible without affecting data privacy. This approach ensures the privacy of sensitive data by enabling the IoT devices to train locally and by sharing only the model updates, which makes it particularly applicable to IoT applications where the involved IoT devices may process personal or confidential data. One of the major strengths of the proposed model is the integration of both differential privacy and homomorphic encryption that together reinforce data security during training. Differential privacy means that each point of individual data is hidden by adding Gaussian noise to local updates to the model and obscures the details of any device data but can still give helpful information on how the model can be improved. In the meantime, the homomorphic encryption will enable aggregation of these updates by the central server without decryption of the updates. This two level of security is new to FL applications related to intrusion detection in IoT and remains a benchmark on privacy-preserving FL framework. Moreover, the lightweight neural network architecture is dedicated to be computationally efficient by design, so it will be suited to resource-constrained IoT devices. It will allow monitoring intrusions in real-time and promising low load on the device performance. Enhancing the novelty by using auditing mechanisms to assure that privacy standards are adhered to, adds the aspect of increased visibility and accountability throughout the learning process. This framework is flexible, open to higher levels of encryption mechanisms, and can dynamically revise privacy budgets, which will provide robustness with changing IoT security demands.

### 4. Results and Discussions

The proposed model is coded using python and executed on windows operating system with the following configuration: Intel® Core i7-1250U Processor (12 M Cache, up to 4.70 GHz) 6 GB memory. It encourages the lightweighted neural network model and the privacy-preserving improvements, including the differential privacy and the homomorphic encryption, components of the used model. This enables the selection of Python as the language of application development based on the libraries needed to implement FL, data encryption, and differential privacy, and comfortable navigation among the issues in the

**Table 1**  
Classification Performance with Confidence Intervals.

| Attack Type          | Accuracy (%) | Precision (%) | Recall (%) | F1-Score (%) |
|----------------------|--------------|---------------|------------|--------------|
| DDoS                 | 92.5 ± 0.8   | 94.2 ± 0.6    | 91.1 ± 0.9 | 92.6 ± 0.7   |
| Brute Force          | 89.3 ± 1.0   | 90.1 ± 0.9    | 88.6 ± 1.1 | 89.3 ± 0.9   |
| Spoofing             | 87.7 ± 0.9   | 89.0 ± 1.0    | 85.5 ± 1.2 | 87.2 ± 1.1   |
| DoS                  | 94.1 ± 0.7   | 95.6 ± 0.5    | 93.4 ± 0.8 | 94.5 ± 0.6   |
| Recon                | 90.8 ± 0.8   | 91.2 ± 0.9    | 90.1 ± 0.7 | 90.7 ± 0.8   |
| Web-based            | 86.9 ± 1.1   | 88.3 ± 1.2    | 85.4 ± 1.3 | 86.8 ± 1.2   |
| Mirai                | 93.6 ± 0.6   | 94.0 ± 0.7    | 92.8 ± 0.8 | 93.4 ± 0.7   |
| Malware              | 85.2 ± 1.3   | 86.3 ± 1.2    | 84.0 ± 1.4 | 85.1 ± 1.3   |
| SQL Injection        | 88.7 ± 1.0   | 89.6 ± 0.9    | 87.5 ± 1.1 | 88.5 ± 1.0   |
| Cross-Site Scripting | 89.1 ± 0.9   | 90.5 ± 1.0    | 87.8 ± 1.1 | 89.1 ± 0.9   |

processing terms. This was as the build and development was average in the RAM whose capacity was 6 GB in order to utilize the capabilities of Python to manage the memory and maximize the computation to detect intrusions. The working principle of Privacy preserving FL for IoT intrusion detection is referred to a concept of distributed approach in which several IoT devices can train a machine learning model simultaneously without sending their input data to the centralized server. Each device does not transfer data to access the local level and instead train a local level model locally. Therefore, the parameters are only transferred to a central server, not that data—including weights and bias—after a local training. Such a structure reduces the risk of any data leak by a significant margin because Props information remains on different devices rather than being routed through hubs or being transmitted via networks. The server subsequently averages these updates and creates a global model that concludes the knowledge to the data without breaching privacy of the data. This approach is quite appropriate within the IoT field because data is often defined as the personally identifiable information and, thus, its privacy is to be secured.

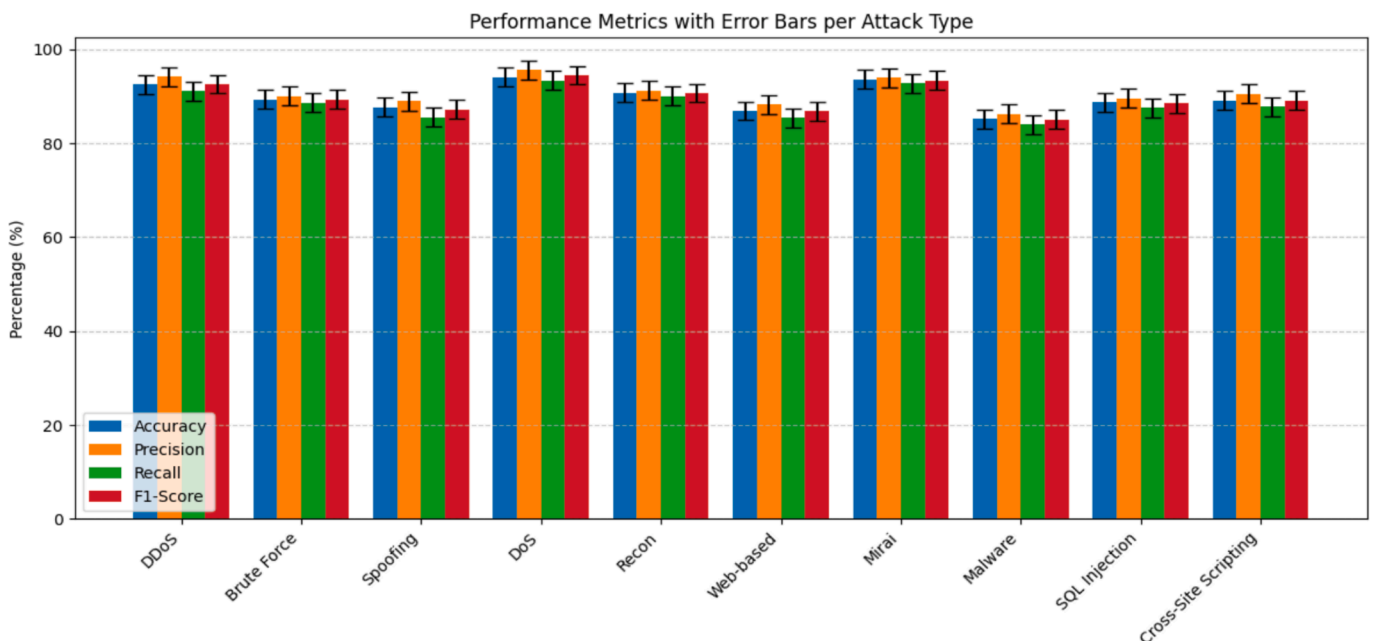
Differential privacy is integrated in the structure of FL in addition to privacy enhancement. Differential privacy can then work by injecting noise to each of the device model updates that are transmitted to the server. This noise disallows direct detection of certain characteristics and patterns in the synthesized model that way a separate specific data could not be isolated in the received outcomes. To optimize the model accuracy against the data privacy, during FL, a privacy parameter

termed privacy budget is present, which regulates the extent to which noise is added. Privacy budget demonstrates the extent to which the gadget information is covert, and the smaller figure ensures privacy. Differential privacy is thus a good statistical camouflage that allowed each device to make some contributions to the global model, rather than prying open the secrets.

Table 1 and Fig. 6 provides accuracy, precision, recall, F1 score and supporting figures for various types of attacks categorized under different types of cyberattacks. The support column gives the actual counts per each attack type to help understand each of the calculated metrics. In the case of DDoS attacks the proposed model yields 92.5 % classification accuracy accompanied by high precision, 94.2 %, recall, 91.1 % and an F1 score of 92.6 %. Regarding the metrics achieved by DoS attacks, the highest accuracy of 94.1 % alongside with an F1-score of 94.5 % indicates that the presented model has high accuracy in the matter of detecting these attacks. Again, both in Brute Force and Recon attacks, the model performance is nearly equivalent with an accuracy of 89.3 % and 90.8 % respectively. For the Brute Force, the F1-score found is 89.3 %, meanwhile, for Recon is 90.7 % suggesting that they are accurate in their detection. Web-based, Spoofing, and SQL Injection types of attacks are slightly less accurate. For instance, the accuracy of Spoofing is 87.7 % while the F1 – score is 87.2 %. Out of the four web-based attacks, we have 86.9 % accuracy and 86.8 F1-score indicating

**Table 2**  
Model Performance with and without Differential Privacy.

| Privacy Budget (e)          | Accuracy (%) | Precision (%) | Recall (%) | F1-Score (%) | Computation Time (ms) |
|-----------------------------|--------------|---------------|------------|--------------|-----------------------|
| No DP                       | 94.2         | 95.1          | 93.6       | 94.3         | 120                   |
| $\epsilon = 0.5$            | 91.4         | 92.3          | 90.2       | 91.2         | 150                   |
| $\epsilon = 1.0$            | 92           | 92.8          | 91.3       | 92           | 145                   |
| $\epsilon = 2.0$            | 93.1         | 94            | 92.2       | 93.1         | 130                   |
| $\epsilon = 5.0$            | 93.7         | 94.5          | 92.8       | 93.6         | 125                   |
| $\epsilon = 10.0$           | 93.9         | 94.6          | 93         | 93.8         | 122                   |
| $\epsilon = 20.0$           | 94           | 94.8          | 93.4       | 94           | 121                   |
| $\epsilon = 50.0$           | 94.1         | 95            | 93.5       | 94.3         | 120                   |
| $\epsilon = 100.0$          | 94.1         | 95            | 93.6       | 94.4         | 120                   |
| $\epsilon = \infty$ (No DP) | 94.2         | 95.1          | 93.6       | 94.3         | 120                   |



**Fig. 6.** Model Accuracy and Metrics for Different Attack Types.

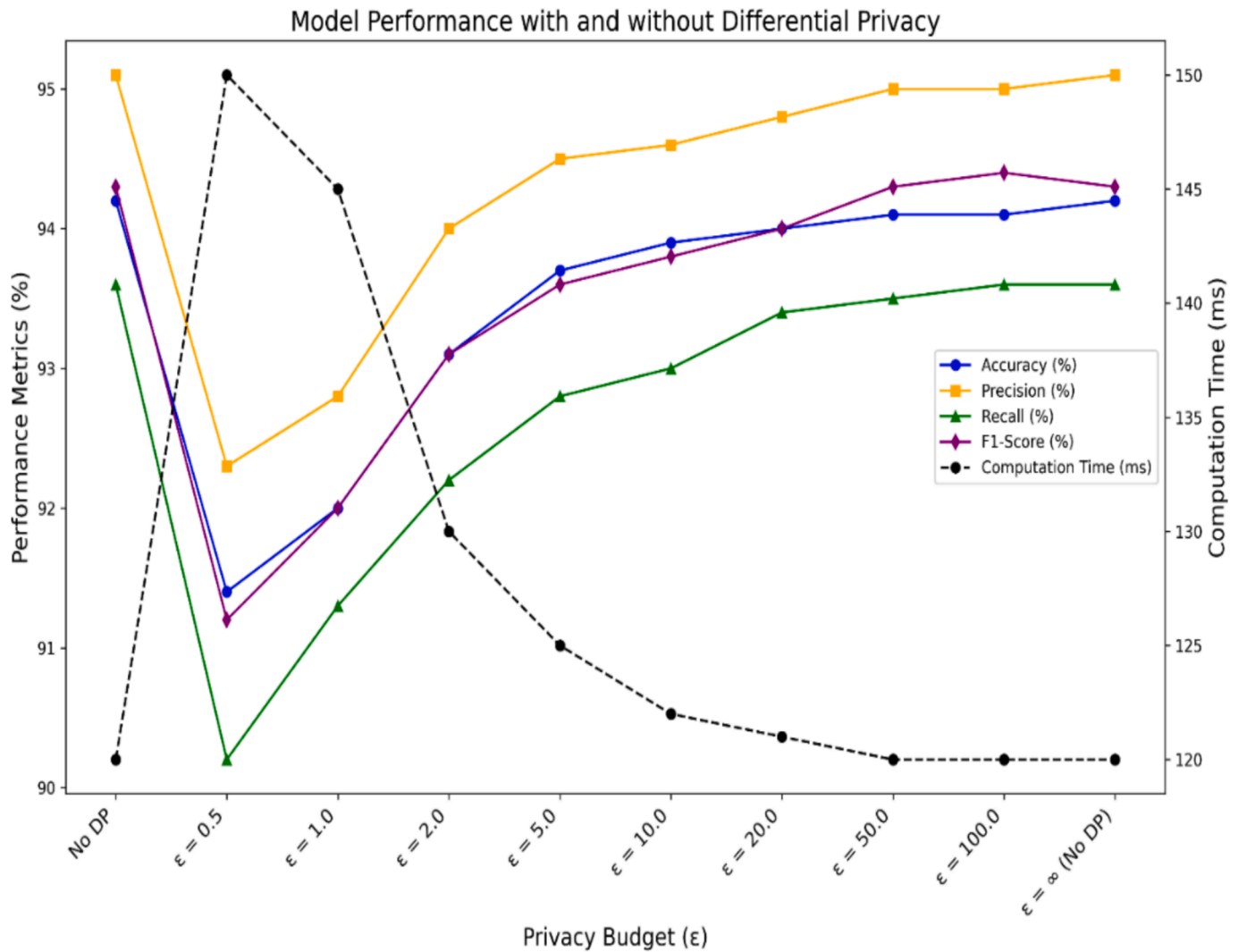


Fig. 7. Model Performance with and without Differential Privacy.

that the results are moderately reliable for these attack types. Sensitivity of Mirai attacks is achieved 93.6 %, specificity is as high as 94 %, and recall value is as low as 92.8 % for which the F1-score of Mirai attacks is 93.4 %. Nonetheless, attack vectors such as Malware and Cross-Site Scripting are less accurate with, respectively, 85.2 % and 89.1 % of accuracy while both scanned, as noted, the F1-scores within the range of 85.1 % and 89.1 %. In general, all the attack types are detected with high accuracy, but the best results were achieved for DDoS, DoS, and Mirai attacks, thus, the proposed model might be most effective in situations when a wide range of cyber threats characterizes the environment.

Table 2, and Fig. 7 shows a comparison of model performance –Accuracy, Precision, Recall, F1-score, computation time, and for Differential Privacy, DP with different privacy budget ( $\epsilon$ ). The conclusion, based upon the results, concerns the fact that using differential privacy leads to the trade-off between accuracy and privacy and the results get progressively better as the privacy budget rises. Without differential privacy ( $\epsilon = \infty$ ), the model has the accuracy of 94.2 %, a precision of 95.1 %, a recall of 93.6 %, and F1-score of 94.3 %, with the computation time of 120 ms. When DP is introduced with setting the privacy budget  $\epsilon = 0.5$ , the accuracy is reduced to 91.4 % and computation time is increased to 150 ms, which shows the potential cost of using high levels of privatisation. However, we observe that the model is still accurate in its predictions with overall accuracy of 92.3 % and F1-score of 91.2 %. If the privacy budget increases, performance metrics rise up gradually. However, by setting  $\epsilon = 2.0$ , accuracy increases to 93.1 %, while the

computation time reduces to 130 ms, indicating a good trade-off between privacy and accuracy. When  $\epsilon = 5.0$  then accuracy is outweighed at about 93.7 %, and the time taken to make the calculations is slightly less than 125 ms. When value of  $\epsilon$  becomes 20.0, then difference between DP Model and non-DP model is only 0.2 % and computation time is 121 ms. At higher epsilon values (e.g.,  $\epsilon = 50.0$ ,  $\epsilon = 100.0$ ), the accuracy level remains at 94.1 %, and all the performance metrics are close to the non-DP model with computation time at 120 ms. These results imply that lower privacy constraints (higher  $\epsilon$  values) influence both accuracy and computation time, but moderate to high privacy budgets are ideal since they offer near-original performance with a slight computation cost increment.

Homomorphic encryption builds up an additional layer of privacy by again encrypting updated models as they are transmitted from a device. This encryption method even enables performing mathematical operations on the data, which have been encrypted; hence, the central server can combine other encrypted model updates without having to decrypt them. Consequently, the server can achieve the same goal of constructing a global model without ever receiving encrypted updates. This makes sure that even in case of server compromise; individual model updates persist safely. Homomorphic encryption is very useful when it comes to the cases when the protection of data is priority since it will remain protected even if it is schemed in an untrusted setting. However, because of its high computational cost, this kind of encryption is usually implemented in a lightweight manner in IoT settings.

**Table 3**

Model Performance with and without Homomorphic Encryption.

| Encryption Level     | Accuracy (%) | Precision (%) | Recall (%) | F1-Score (%) | Computation Time (ms) |
|----------------------|--------------|---------------|------------|--------------|-----------------------|
| No Encryption        | 94.1         | 95            | 93.4       | 94.2         | 120                   |
| Basic Encryption     | 94           | 94.9          | 93.3       | 94.1         | 130                   |
| Medium Encryption    | 93.8         | 94.6          | 93         | 93.7         | 140                   |
| High Encryption      | 93.6         | 94.4          | 92.8       | 93.5         | 160                   |
| Full Homomorphic     | 93.5         | 94.3          | 92.7       | 93.4         | 200                   |
| Optimized Encryption | 94.1         | 94.9          | 93.4       | 94.2         | 125                   |

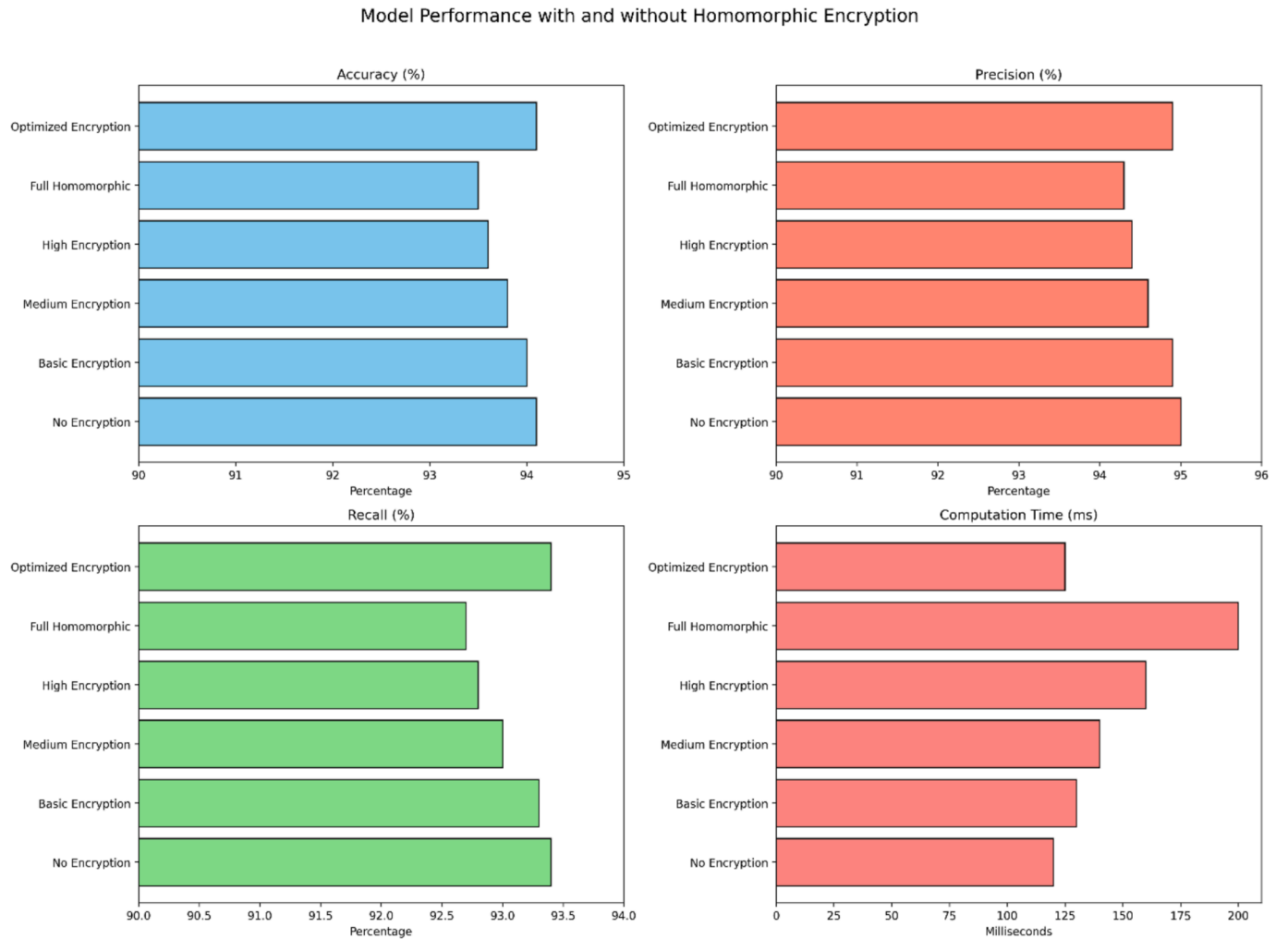
**Fig. 8.** Model Performance with and without Homomorphic Encryption.

Table 2 shows that there is a measurable privacy-accuracy trade-off to apply differential privacy. At a tight privacy budget ( $\epsilon = 0.5$ ), the accuracy reduced to 91.4 % and latency increased to 150 ms as opposed to 94.2 % accuracy and 120 ms latency in the non-DP case. At intermediate budgets ( $\epsilon = 2.0-5.0$ ), however, the accuracy reached a plateau of between 93.1 and 93.7 %, corresponding to a very moderate (at 7–10 ms) latency overhead. This indicates that moderate values of  $\epsilon$  guarantee almost full-baseline detection accuracy, but with justifiable privacy guarantees and are thus feasible in real-time IoT production scenarios. In the same spirit, Table 3 shows the implications of homomorphic encryption computationally. The targeted baseline (no encryption) had an accuracy of 94.1 % with the latency of 120 ms whereas full homomorphic encryption retained about the same accuracy of 93.5 % but consumed 200 ms latency (66.7 % overhead). Encryption was optimised to fill this gap down to 94.1 of accuracy at a latency of 125 ms, being just 4.2 % overhead. These results verify that although encryption strength

can directly affect the computation, lightweight optimization can fight the penalty of latency without compromising data protection.

Table 3 and Fig. 8 exhibits the overall performance measures such as accuracy, precision, recall, F1-measure, and the time taken for computation with an increase in homomorphic encryption level. These results raise awareness on how levels of encryption influence performance and computation. The model without encryption gives the test accuracy of 94.1 %, precision of 95 %, recall of 93.4 % and F1-score of 94.2 % with CPU processing time of 120 ms. With a basic encryption applied to the model, the accuracy only drops slightly to 94 %, and there is slight degradation in precision to 94.9 % and recall to 93.3 %, but computation time increases to 130 ms. Algorithms provide smaller increases in performance, and small decreases in efficiency and computation time as encryption continues to develop. In medium encryption, the accuracy is 93.8 % while the time for the computation is 140 ms. Higher encryption brings the accuracy down to 93.6 %, and increases computation time to

**Table 4**

Impact of Auditing Mechanisms on Compliance and Detection.

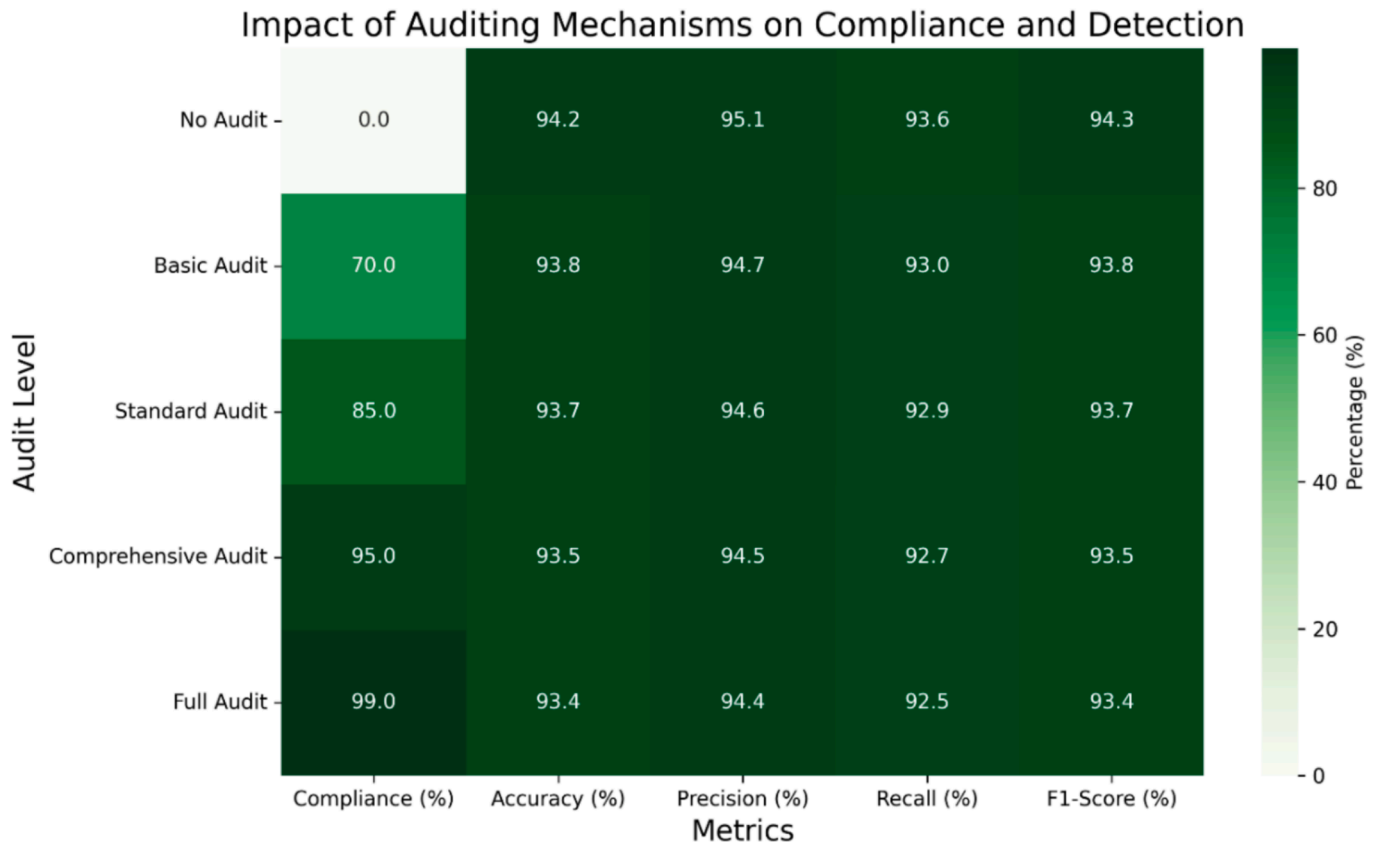
| Audit Level         | Compliance (%) | Accuracy (%) | Precision (%) | Recall (%) | F1-Score (%) |
|---------------------|----------------|--------------|---------------|------------|--------------|
| No Audit            | 0              | 94.2         | 95.1          | 93.6       | 94.3         |
| Basic Audit         | 70             | 93.8         | 94.7          | 93         | 93.8         |
| Standard Audit      | 85             | 93.7         | 94.6          | 92.9       | 93.7         |
| Comprehensive Audit | 95             | 93.5         | 94.5          | 92.7       | 93.5         |
| Full Audit          | 99             | 93.4         | 94.4          | 92.5       | 93.4         |

160 ms. Finally, at the highest level using full homomorphic encryption, both accuracy and F1-score are 93.5 % and 93.4 % correspondingly. The computation time of fully encrypted operations is clearly a challenge with 200 ms. Optimized encryption, which seeks to achieve a good level of security and still take lesser time to execute, is as good as the no encryption with an accuracy of 94.1 %, precision, of 94.9 %, recall of 93.4 % and an F1-score of 94.2 % though the computation time is slightly higher, taking 125 ms to complete the process. These results indicate that while increasing the degree of encryption degrades computation speed, improved encryption strategies can here preserve

the efficiency of a model without significantly slowing computation, thus presenting a secure solution.

Auditing technologies have been incorporated in the FL process to check that privacy preservation techniques are rightly implemented and used across the board. Monitoring records updates on each device; noise levels, encryption status, as well as privacy budget conformity. These logs are retained and scrutinized from time to time to ensure that the devices operate according to the previously set baseline privacy levels, including maximum noise levels for differential privacy, and the correct encryption modes to be used. An essential aspect accompanying auditing mechanisms or equivalent is that it adds a layer of trust and a means to check whether a specific party follows the rules or it is potentially violating privacy while participating in FL.

Table 4 and Fig. 9 shows how varying levels of auditing affect compliance and all measures of model performance including accuracy, precision, recall, and F1-score. Utilizing the data it is possible to consider the fact that higher auditing levels contribute to increased levels of compliance and at the same time have a minimal impact in performance indicators. If no audit mechanism is used, it results in 0 % compliance; however, the model yields the best results of all performance indicators, with the accuracy of 94.2 %, precision of 95.1 %, recall of 93.6 % and F1-score of 94.3 %. By raising compliance to 70 % through the

**Fig. 9.** Impact of Auditing Mechanisms on Compliance and Detection.**Table 5**

Model Training Performance across Different IoT Device Types.

| Device Type        | CPU (GHz) | Memory (MB) | Accuracy (%) | Computation Time (ms) | Energy Consumption (mWh) |
|--------------------|-----------|-------------|--------------|-----------------------|--------------------------|
| Low-end Sensor     | 0.5       | 128         | 91.5         | 200                   | 30                       |
| Standard Sensor    | 1         | 256         | 92.7         | 180                   | 25                       |
| Edge Device        | 1.5       | 512         | 93.8         | 150                   | 20                       |
| Smart Camera       | 2         | 1024        | 94.1         | 130                   | 18                       |
| Gateway Device     | 2.5       | 2048        | 94.3         | 120                   | 15                       |
| High-end Processor | 3         | 4096        | 94.5         | 110                   | 12                       |

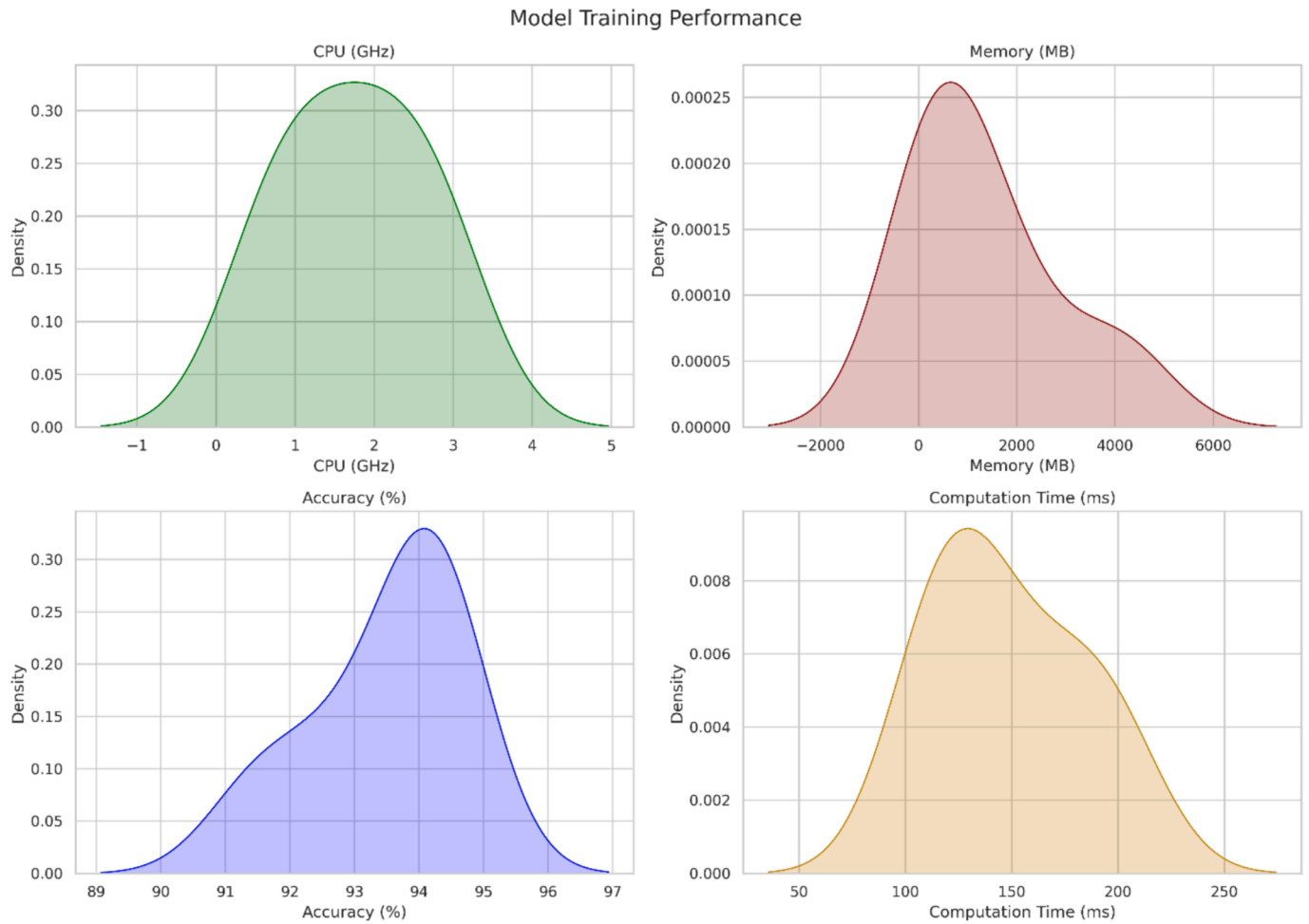


Fig. 10. Model Training Performance.

introduction of a basic audit, there are minor model backdrops, which, in this case, give an accuracy of 93.8 % as well as an F1-score of 93.8 %. As the level of auditing gains the higher level, the level of compliance raises, but effectiveness indicators become lower. In the case of the standard audit, the compliance percentage is 85 %, using metrics, the accuracy is 93.7 %, and F1-score is 93.7 %. When it comes to compliance, compliance rises even more to 95 % in the comprehensive audit, with the model achieving 93.5 % accuracy when using F1-score and balancing between compliance and performance. The highest audit level, the full audit, achieves near-total compliance at 99 %, but results in a slight decrease in performance metrics: 93.4 % accuracy, 94.4 % precision, 92.5 % recall, and an F1-score of 93.4 %. Overall, these findings imply that while enhanced auditing structures yield a considerable boost to compliance levels, they do so at close to zero cost to model accuracy, affording evidence of the possibilities of incorporating robust auditing techniques into the process.

Table 5 and Fig. 10 shows a comparison of the speed, memory, accuracy, computational time, and energy of the IoT devices used for training models to validate the proposed approach. This information demonstrates the role of hardware characteristics as determinants of performance and energy utilization. The low-end sensor with 0.5 GHz of CPU and 128 MB of memory reports a 91.5 % accuracy in facial recognition yet requires 200 ms of CPU time and 30mWh of energy consumption. Increasing the generality of the device, its performance and efficiency increase simultaneously. For instance, the standard sensor, a diagram with 1 GHz CPU, 256 MB memory, gives 92.7 % accuracy at a computation time of 180 ms with energy consumption of 25mWh. According to our results, edge devices with a 1.5 GHz CPU and

Table 6

Model Performance under Different Data Splits.

| Training/<br>Testing<br>Ratio | Accuracy<br>(%) | Precision<br>(%) | Recall<br>(%) | F1-<br>Score<br>(%) | Training<br>Time (s) |
|-------------------------------|-----------------|------------------|---------------|---------------------|----------------------|
| 50/50                         | 92              | 92.8             | 91.3          | 92                  | 40                   |
| 60/40                         | 93.1            | 93.9             | 92.1          | 93                  | 45                   |
| 70/30                         | 94.2            | 95               | 93.4          | 94.2                | 50                   |
| 80/20                         | 94              | 94.9             | 93.3          | 94.1                | 55                   |
| 90/10                         | 93.6            | 94.5             | 92.8          | 93.5                | 60                   |

512 MB memory yield high efficiency and accuracy with 93.8 % accuracy, 150 ms computation time and 20mWh energy consumption. Smart cameras used in this work have a 2 GHz CPU and 1024 MB memory and exhibit an accuracy of 94.1 %, along with even lesser computation time, which is 130 ms and energy consumption of 18mWh only. At the higher level of complexity of gateway devices with 2.5 GHz CPU and 2048 MB memory we obtain 94.3 % accuracy with \$120 ms\$ computation time and 15mWh energy consumption. The maximum performance of 94.5 % accuracy, minimum computation time of 110 ms, and lowest energy consumption of 12mW is recorded at 3 GHz CPU & 4096 MB memory. These outcomes showed that greater model capabilities improve model performance and density, making the devices perfect for power-hungry IoT applications that require high accuracy, speed, and efficiency.

Table 6 and Fig. 11 shows the accuracy, precision, recall, F1-score, and the training time of each model depending on the training/testing data split. This piece of data shows the effect of changing the training/testing ratio to the classification accuracy and the training time. On a

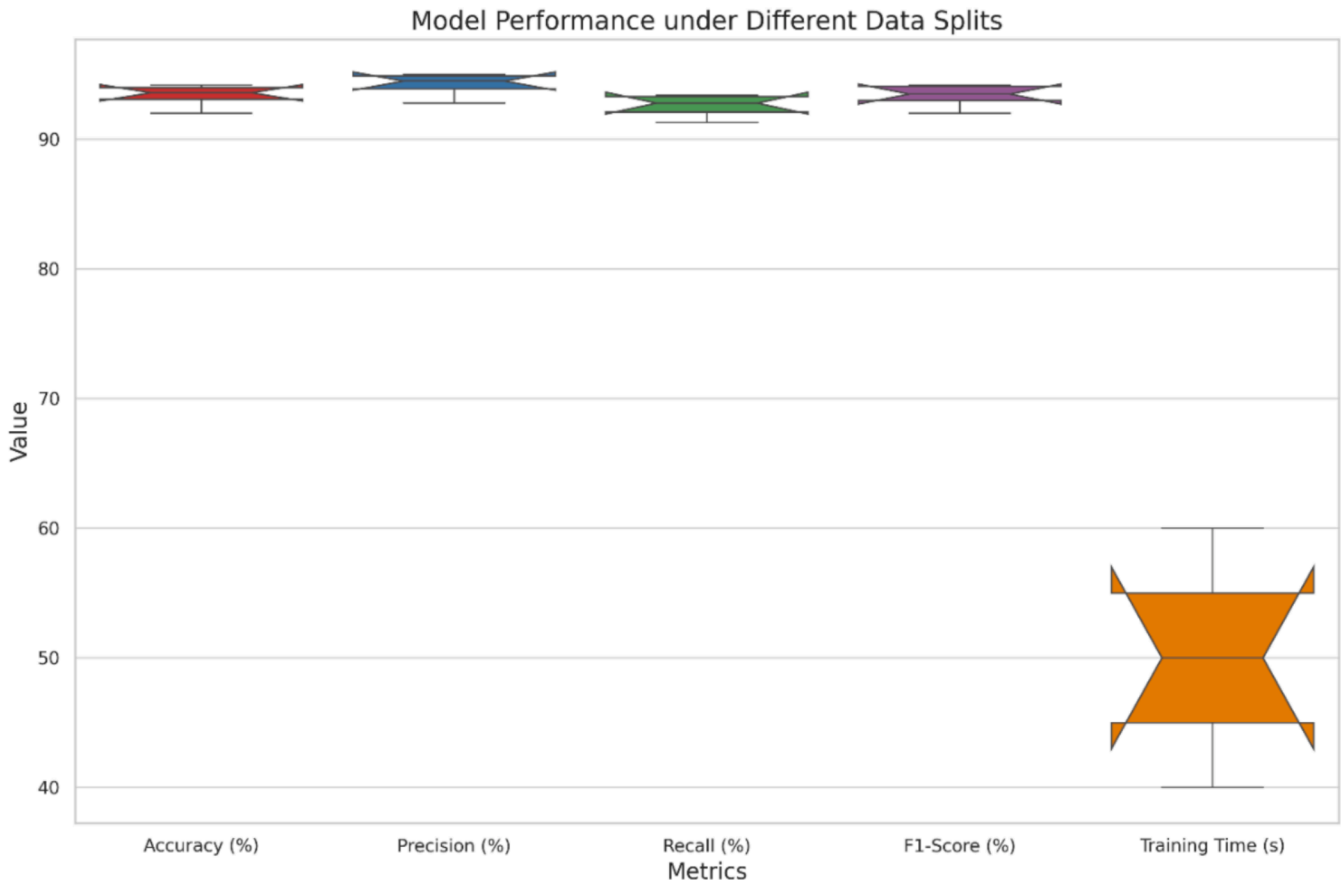


Fig. 11. Model Performance under Different Data Splits.

**Table 7**  
Attack Detection Latency across Different Intrusion Types.

| Attack Type          | Detection Latency (ms) | Accuracy (%) | Precision (%) | Recall (%) | F1-Score (%) |
|----------------------|------------------------|--------------|---------------|------------|--------------|
| DDoS                 | 100                    | 92.5         | 94.2          | 91.1       | 92.6         |
| Brute Force          | 110                    | 89.3         | 90.1          | 88.6       | 89.3         |
| Spoofing             | 120                    | 87.7         | 89            | 85.5       | 87.2         |
| DoS                  | 90                     | 94.1         | 95.6          | 93.4       | 94.5         |
| Recon                | 115                    | 90.8         | 91.2          | 90.1       | 90.7         |
| Web-based            | 125                    | 86.9         | 88.3          | 85.4       | 86.8         |
| Mirai                | 105                    | 93.6         | 94            | 92.8       | 93.4         |
| Malware              | 130                    | 85.2         | 86.3          | 84         | 85.1         |
| SQL Injection        | 100                    | 88.7         | 89.6          | 87.5       | 88.5         |
| Cross-Site Scripting | 125                    | 89.1         | 90.5          | 87.8       | 89.1         |

50/50 train, the model has 92 % accuracy, 92.8 % precision, 91.3 % recall, 92 % F1 score with 40 s of training time. With the training data portion increased to 60 %, the accuracy is 93.1 %, with a precision of 93.9 % and a training time of 45 s, thus slightly better than the previous experiment. Lastly, at a 70/30 split, the highest accuracy is achieved at 94.2 % with a precision of 95 % and recall of 93.4 % and the F1-score of 94.2 % but the training time is high at 50 s. Similar high performance is achieved with equal ratio of data split as 80/20, with a 94 % accuracy, 94.9 % precision and 55 s for training, it is concluded that the size of training is crucial for the accuracy rate. For 90/10 split of training and testing, the accuracy is found to be 93.6 %, precision is 94.5 %, F1 score is 93.5 %, and the training time is observed to be 60 s. These results show that a 70/30 split offers the most reasonable training time as it offers the

closest range for the maximum achievable accuracy alongside F1-Score. This split is appropriate for situations where difficult to achieve high performance is important, but it is acceptable if the size of data slightly varies.

Table 7 and Fig. 12, and Fig. 13 present attack detection latency, accuracy, precision, recall, and F1-score depending on different intrusion types. Detection latency is expressed in milliseconds (ms) as the time needed to detect each attack type while performance measures contribute to detection quality. For DDoS, the detection latency is reasonable at 100 ms with good detection accuracy at 92.5 % and reasonable precision at 94.2 % as well as high recall at 91.1 % and an F1 score of 92.6 %. The average response time for DoS attacks is considerably lower at 90 ms to accomplish the highest performance-related criteria of 94.1 % accuracy and an F1-score of 94.5 %. With regard to Brute Force attacks the mean detection latency amounts to 110 ms, and the mean accuracy is 89.3 %, while the F1 score is 89.3 %. Spoofing attacks take 120 ms for detection, with 87.7 % accuracy and 87.2 % F-score, meaning moderate detection efficiency. The overall performance of Recon attacks is measured with a latency of 115 ms and accuracy as well as F1-score equal to 90.8 % while the various Web-based attacks need 125 ms for classification and have slightly worse classification criteria, with an accuracy of 86.9 % and an F1-score score of 86.8 %. A 105 ms time limit is used to detect Mirai attacks with 93.6 % accuracy and an F1-score of 93.4 %, and it demonstrates high detection efficiency. Overall, recognizing Malware and SQL Injection attacks has slightly higher latencies: 130 ms and 100 ms correspondingly but lower accuracy: 85.2 % and 88.7 % correspondingly. Cross-Site Scripting detection is described, with 125 ms of time, 89.1 % of accuracy, and 89.1 % F1-score. In general, the results of DoS and DDoS attacks demonstrate both low latency and high performance, while longer times needed for detecting the attacks of Malware and Web-based type with somewhat

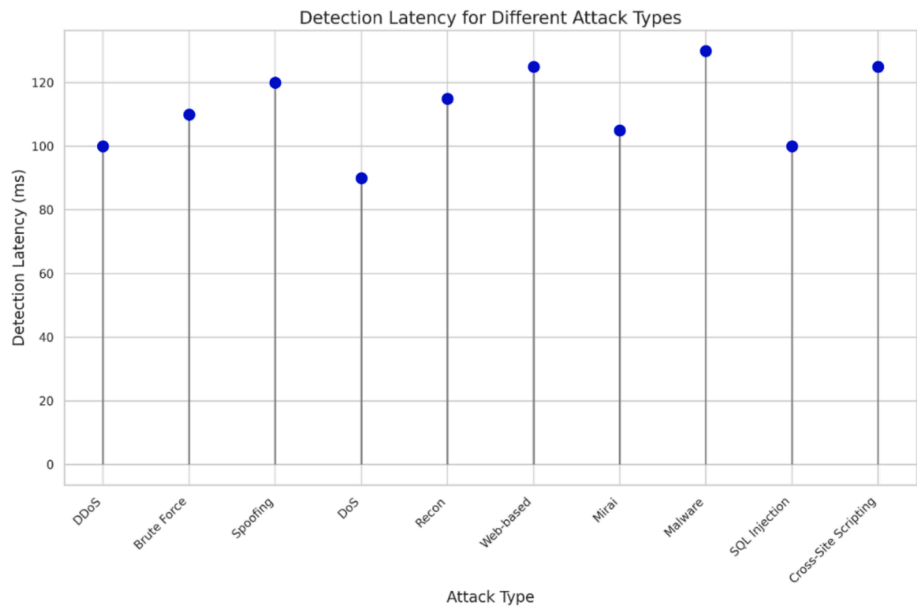


Fig. 12. Detection Latency for Different Attack Types.

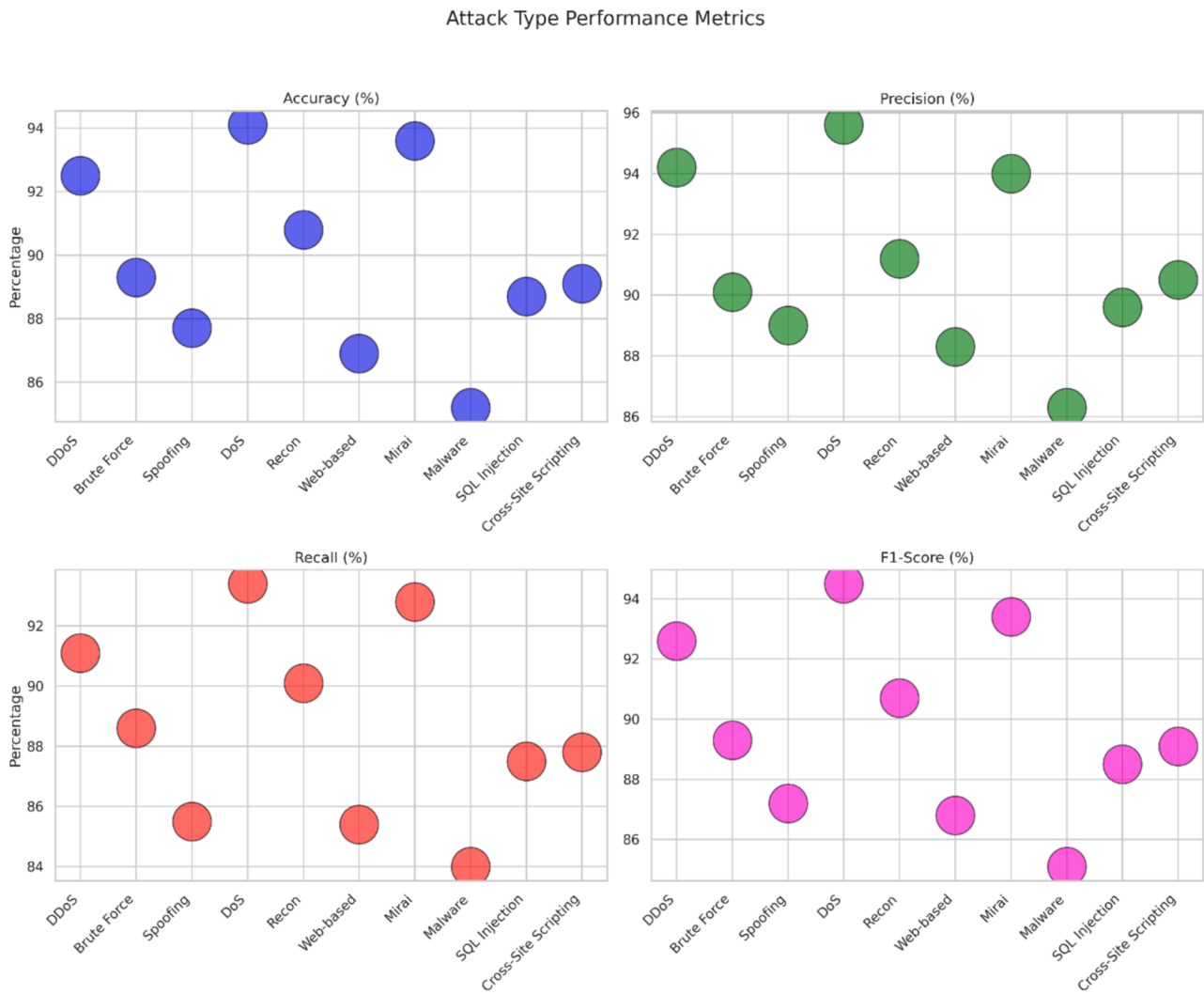


Fig. 13. Attack Type Performance Metrics.

lower accuracy. This data indicates that detection efficiency is highly dependent on the intrusion type and the faster the detection, the better in many circumstances.

The proposed framework shows good results in several types of intrusion; however, its application in large-scale, heterogeneous IoT environment deserves further research. Deployments of IoT systems in the real world commonly deal with thousands of devices having different processing power, different communications protocols, and energy limitations, which could increase the computational and communications overheads that are found in this paper. Scalability is also an important factor, especially when there is high frequency at the global level in updating the global model, which may strain the bandwidth as well as cause delays in the synchronization. The future extensions will consider the implementation of multi-tier FL strategies (e.g., edge-fog-cloud hierarchies) to alleviate scalability issues, and enable a wide range of devices to support the ecosystem.

## 5. Conclusion and future work

In this paper, we proposed a security privacy-preserving model of federated learning of IoT intrusion detection that used differential privacy, homomorphic encryption and auditing as part of the lightweight neural model. The protocol allows a collaborative training paradigm without exchanging raw traffic and secures the information level of training nodes in local learning and centralized aggregation.

The model obtained 93.5 % overall accuracy on the IoT Intrusion Detection data set (1,191,264 instances, 47 features) detecting a range of various attacks, such as DDoS, Brute Force, Spoofing, Recon, and Mirai. In terms of attack-wise analysis, the attributes revealed sufficiently low latency and high accuracy on DoS/DDoS (e.g., DoS 90 ms, 94.1 %; DDoS 100 ms, 92.5 %), but moderate on the Web-based/Malware attacks (e.g., Web 125 ms, 86.9 %).

Differential privacy had a manageable trade-off: at  $\epsilon = 0.5$  the accuracy/latency was 91.4/150 ms, but  $\epsilon = 2-5$  produced 93.1–93.7/1–10 ms as compared to no-DP (94.2 %, 120 ms). Homomorphic encryption did not compromise accuracy ( $\leq 0.6$  % delta) but presented a latency of 120 ms (no-HE) to 140–200 ms (depending on level); lightweight schemes decrease this difference. These findings determine  $\epsilon = 2$  and optimized HE as feasible IoT operating conditions in real-time security.

Scalability with heterogeneous compute and sporadic links is an open problem; future work should demonstrate scalable hierarchical/asynchronous FL with partial participation to alleviate the stragglers and bandwidth spikes. The advantages of privacy-utility will also be elucidated by comparative baselines against centralized and non-FL distributed IDS. Finally, dynamic privacy budgets varying  $\epsilon$  per attack regime and less heavy HE/SMPC could ensure stronger guarantees without overstepping the strict latency limits used in the per-attack intuition.

## CRedit authorship contribution statement

**A. Puviasaru:** Writing – review & editing, Writing – original draft, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **V.K Sudha:** .

## Funding

Authors declare no funding for this research.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgements

Not applicable.

## References

- [1] Iacovos Ioannou, et al., (2024), "GEMILDS-MIOT: A Green Effective Machine Learning Intrusion Detection System based on Federated Learning for Medical IoT network security hardening", CC, Volume 218, Pages 209-239, ISSN 0140-3664, DOI: 10.1016/j.comcom.2024.02.023..
- [2] Kun Huang, et al., (2024), "A comprehensive intrusion detection method for the internet of vehicles based on federated learning architecture", CS, Volume 147, 104067, ISSN 0167-4048, DOI: 10.1016/j.cose.2024.104067.
- [3] Radjaa Bensaid, et al., (2024), "Securing fog-assisted IoT smart homes: a federated learning-based intrusion detection approach", CC 28, 50, DOI: 10.1007/s10586-024-04711-0.
- [4] Mahadik SS, et al. Edge-Federated Learning-Based Intelligent Intrusion Detection System for Heterogeneous Internet of Things. Access 2024;12:81736–57. <https://doi.org/10.1109/ACCESS.2024.3410046>.
- [5] Sun G, Li Y, Liao D, Chang V. Service Function Chain Orchestration Across Multiple Domains: A Full Mesh Aggregation Approach. IEEE Transactions on Network and Service Management Sept. 2018;15(3):1175–91. <https://doi.org/10.1109/TNSM.2018.2861717>.
- [6] Sun G, Zhang Y, Liao D, Yu H, Du X, Guizani M. Bus-Trajectory-Based Street-Centric Routing for Message Delivery in Urban Vehicular Ad Hoc Networks. IEEE Transactions on Vehicular Technology Aug. 2018;67(8):7550–63. <https://doi.org/10.1109/TVT.2018.2828651>.
- [7] Liu Y, Li W, Dong X, Ren Z. Resilient Formation Tracking for Networked Swarm Systems Under Malicious Data Deception Attacks. International Journal of Robust and Nonlinear Control April 2025;35(6):2043–52. <https://doi.org/10.1002/rnc.7777>.
- [8] Nawaz RAB. MUHAMMAD KAMRAN JAMIL, Farwa Asmat, Muhammad Azeem, and Bandar Almohsen, "A novel application by computing partition dimension of subdivision of circulant network". IEEE Access 2023;11. <https://doi.org/10.1109/ACCESS.2024.0429000>.
- [9] Xiangyu Zou, Jingsong Yuan, Philip Shilane, Wen Xia, Haijun Zhang, and Xuan Wang. 2022. From Hyper-dimensional Structures to Linear Structures: Maintaining Deduplicated Data's Locality. ACM Trans. Storage 18, 3, Article 25 (August 2022), 28 pages. Doi: 10.1145/3507921.
- [10] Wen Xia, Lifeng Pu, Xiangyu Zou, Philip Shilane, Shiyi Li, Haijun Zhang, and Xuan Wang. 2022. The Design of Fast and Lightweight Resemblance Detection for Efficient Post-Deduplication Delta Compression. ACM Trans. Storage 19, 3, Article 22 (August 2023), 30 pages. Doi: 10.1145/3584663.
- [11] Yu Yao, Feng Shu, Xu Cheng, Haitao Liu, Pu Miao, and Lenan Wu. 2023. Automotive Radar Optimization Design in a Spectrally Crowded V2I Communication Environment. Trans. Intell. Transport. Sys. 24, 8 (Aug. 2023), 8253–8263. Doi: 10.1109/TITS.2023.3264507.
- [12] Zhou W, Xia C, Wang T, Liang X, Lin W, Li X, et al. 104155. ISSN 2025;0167–4048. <https://doi.org/10.1016/j.cose.2024.104155>.
- [13] Li H, Xia C, Wang T, Wang Z, Cui P, Li X. GRASS: Learning Spatial–Temporal Properties From Chainlike Cascade Data for Microscopic Diffusion Prediction. IEEE Transactions on Neural Networks and Learning Systems Nov. 2024;35(11): 16313–27. <https://doi.org/10.1109/TNNLS.2023.3293689>.
- [14] Yukun Cheng, Xiaotie Deng, Yuhao Li, Xiang Yan, "Tight incentive analysis of Sybil attacks against the market equilibrium of resource exchange over general networks", Games and Economic Behavior, Volume 148, 2024, Pages 566-610, ISSN 0899-8256, Doi: 10.1016/j.geb.2024.10.009.
- [15] Begum K, et al. BFLIDS: Blockchain-Driven Federated Learning for Intrusion Detection in IoMT Networks. Sensors 2024;24(14):4591. <https://doi.org/10.3390/s24144591>.
- [16] Thein TT, et al. "Personalized federated learning-based intrusion detection system: Poisoning attack and defense", FGCS, Volume 153. ISSN 2024;182–192:0167–739. <https://doi.org/10.1016/j.future.2023.10.005>.
- [17] Mohammad Amiri-Zarandi, et al., (2023), "SIDS: A federated learning approach for intrusion detection in IoT using Social Internet of Things", CN, Volume 236, 110005, ISSN 1389-1286, DOI: 10.1016/j.comnet.2023.110005.
- [18] Zhong C, et al. Federated learning-guided intrusion detection and neural key exchange for safeguarding patient data on the internet of medical things. IJML 2024;15:5635–65. <https://doi.org/10.1007/s13042-024-02269-2>.
- [19] F. Xu, H. -C. Yang and M. -S. Alouini, "Energy Consumption Minimization for Data Collection From Wirelessly-Powered IoT Sensors: Session-Specific Optimal Design With DRL," in IEEE Sensors Journal, vol. 22, no. 20, pp. 19886-19896, 15 Oct.15, 2022, doi: 10.1109/JSEN.2022.3205017.
- [20] Song D, Zhao J, Zhu B, Han J, Jia S. Subjective Driving Risk Prediction Based on Spatiotemporal Distribution Features of Human Driver's Cognitive Risk. IEEE Transactions on Intelligent Transportation Systems Nov. 2024;25(11):16687–703. <https://doi.org/10.1109/TITS.2024.3409874>.
- [21] Tan Tang, Rui Dai, Chenguang Zuo, Jingwen Chen, Keqin Li, and Zheng Qin. 2025. A Low-Rate DoS Attack Mitigation Scheme Based on Port and Traffic State in SDN. IEEE Trans. Comput. 74, 5 (May 2025), 1758–1770. Doi: 10.1109/TC.2025.3541143.
- [22] Xiaoguang Liu, Peize Liu, Bo Yang, Yizhan Chen, "One multi-receiver certificateless searchable public key encryption scheme for IoMT assisted by LLM", Journal of

- Information Security and Applications, Volume 90, 2025, 104011, ISSN 2214-2126, Doi: 10.1016/j.jisa.2025.104011.
- [23] Gang Xu, Lele Lei, Yanhui Mao, Zongpeng Li, Xiu-Bo Chen, Kejia Zhang, "CBRFL: A framework for Committee-based Byzantine-Resilient Federated Learning", *Journal of Network and Computer Applications*, Volume 238, 2025, 104165, ISSN 1084-8045, Doi: 10.1016/j.jnca.2025.104165.
- [24] Wu X, Zou B, Lu C, Wang L, Zhang Y, Wang H. Dynamic Security Computing Framework With Zero Trust Based on Privacy Domain Prevention and Control Theory. *IEEE Journal on Selected Areas in Communications* June 2025;43(6): 2266–78. <https://doi.org/10.1109/JSAC.2025.3560036>.
- [25] Liu Y, Dong X, Zio E, Cui Y. Event-Triggered Multiple Leaders Formation Tracking for Networked Swarm System With Resilience to Noncooperative Nodes. In: *IEEE transactions on cybernetics*; 2025. pp.. <https://doi.org/10.1109/TCYB.2025.3580666>.
- [26] Mikail Mohammed Salim, et al., (2024), "Digital Twin and federated learning enabled cyberthreat detection system for IoT network", *FGCS*, Volume 161, Pages 701-713, ISSN 0167-739X, DOI: 10.1016/j.future.2024.07.017.
- [27] Ohtani T, et al. IDAC: Federated Learning-Based Intrusion Detection Using Autonomously Extracted Anomalies in IoT. *Sensors* 2024;24(10):3218. <https://doi.org/10.3390/s24103218>.
- [28] ChandraUmakantham OK, et al. Enhancing Intrusion Detection Through Federated Learning With Enhanced Ghost Binet and Homomorphic Encryption. *IEEE Access* 2024;12:24879–93. <https://doi.org/10.1109/ACCESS.2024.3362347>.
- [29] Raza M, et al. Federated Learning for Privacy-Preserving Intrusion Detection in Software-Defined Networks. *IEEE Access* 2024;12:69551–67. <https://doi.org/10.1109/ACCESS.2024.3395997>.
- [30] Huang J, et al. Improved Intrusion Detection Based on Hybrid Deep Learning Models and Federated Learning. *Sensors* 2024;24(12):4002. <https://doi.org/10.3390/s24124002>.
- [31] Shao J-M, et al. "Automated federated learning for intrusion detection of industrial control systems based on evolutionary neural architecture search", *CS*, Volume 143, ISSN 2024;103910:0167–4048. <https://doi.org/10.1016/j.cose.2024.103910>.
- [32] Qiao Y, Lü J, Wang T, Liu K, Zhang B, Snoussi H. A Multihead Attention Self-Supervised Representation Model for Industrial Sensors Anomaly Detection. *IEEE Transactions on Industrial Informatics* Feb. 2024;20(2):2190–9. <https://doi.org/10.1109/TII.2023.3280337>.
- [33] Zhang M, Wei E, Berry R, Huang J. Age-Dependent Differential Privacy. *IEEE Transactions on Information Theory* Feb. 2024;70(2):1300–19. <https://doi.org/10.1109/TIT.2023.3340147>.
- [34] Hongbo Jiang, Panyi Ji, Taiyuan Zhang, Hangcheng Cao, and Daibo Liu. 2024. Two-Factor Authentication for Keyless Entry System via Finger-Induced Vibrations. *IEEE Transactions on Mobile Computing* 23, 10 (Oct. 2024), 9708–9720. Doi: 10.1109/TMC.2024.3368331.
- [35] Li, Xiangjun & Lu, Zukun & Yuan, Muzi & Liu, Wenxiang & Wang, Feixue & Yu, Yi & Liu, Peiguo. (2024). Tradeoff of Code Estimation Error Rate and Terminal Gain in SCER Attack. *IEEE Transactions on Instrumentation and Measurement*. PP. 1-1. 10.1109/TIM.2024.3406807.
- [36] Chen C, Cui J, Qu G, Zhang J. Write+Sync: Software Cache Write Covert Channels Exploiting Memory-Disk Synchronization. *IEEE Transactions on Information Forensics and Security* 2024;19:8066–78. <https://doi.org/10.1109/TIFS.2024.3414255>.
- [37] Xu Y, Ding L, He P, Lu Z, Zhang J. Meta: A Memory-Efficient Tri-Stage Polynomial Multiplication Accelerator Using 2D Coupled-BFUs. *IEEE Transactions on Circuits and Systems I: Regular Papers* Feb. 2025;72(2):647–60. <https://doi.org/10.1109/TCSI.2024.3461736>.
- [38] Gong Y, Yao H, Liu X, Bennis M, Nallanathan A, Han Z. Computation and Privacy Protection for Satellite-Ground Digital Twin Networks. *IEEE Transactions on Communications* Sept. 2024;72(9):5532–46. <https://doi.org/10.1109/TCOMM.2024.3392795>.
- [39] V. Gowdhaman and R. Dhanapal, "Hybrid deep learning-based intrusion detection system for wireless sensor network", *International Journal of Vehicle Information and Communication Systems*, Vol. 9, No. 3, ISSN: 1471-0242, eISSN: 1741-8208, July 5, 2024pp 239-255, Doi: 10.1504/IJVICs.2024.139627.
- [40] Taki Eddine Toufik Djaidja, et al., (2024), "Federated learning for 5G and beyond, a blessing and a curse- an experimental study on intrusion detection systems", *CS*, Volume 139, 103707, ISSN 0167-4048, DOI: 10.1016/j.cose.2024.103707.
- [41] Zhiguo Qu, et al. FEDSA-ResnetV2: An Efficient Intrusion Detection System for Vehicle Road Cooperation Based on Federated Learning. *IEEE ITJ* 2024;11(18): 29852–63. <https://doi.org/10.1109/JIOT.2024.3410871>.
- [42] Junwei Liang, et al., (2024), "Enhanced collaborative intrusion detection for industrial cyber-physical systems using permissioned blockchain and decentralized federated learning networks", *EAAI*, Volume 135, 108862, ISSN 0952-1976, DOI: 10.1016/j.engappai.2024.108862.
- [43] Raja L, et al. An improved federated transfer learning model for intrusion detection in edge computing empowered wireless sensor networks. *CCPE* 2024;36(24): e8259.
- [44] Lopes F, de Caldas Filho, et al. Botnet Detection and Mitigation Model for IoT Networks Using Federated Learning. *Sensors* 2023;23(14):6305. <https://doi.org/10.3390/s23146305>.
- [45] Chao Fu, Guannan Liu, Kun Yuan, and Junjie Wu. 2025. Nowhere to H<sup>2</sup></sup>IDE: Fraud Detection From Multi-Relation Graphs via Disentangled Homophily and Heterophily Identification. *IEEE Trans. on Knowl. and Data Eng.* 37, 3 (March 2025), 1380–1393. Doi: 10.1109/TKDE.2024.3523107.
- [46] Jingyang Hu, Hongbo Jiang, Siyu Chen, Qibo Zhang, Zhu Xiao, Daibo Liu, Jiangchuan Liu, and Bo Li. 2024. WiShield: Privacy Against Wi-Fi Human Tracking. *IEEE J.Sel. A. Commun.* 42, 10 (Oct. 2024), 2970–2984. Doi: 10.1109/JSAC.2024.3414597.
- [47] Siyu Chen, Hongbo Jiang, Jingyang Hu, Tianyue Zheng, Mengyuan Wang, Zhu Xiao, Daibo Liu, and Jun Luo. 2025. Echoes of Fingerprint: Unveiling POS Terminal Passwords Through Wi-Fi Beamforming Feedback. *IEEE Transactions on Mobile Computing* 24, 2 (Feb. 2025), 662–676. Doi: 10.1109/TMC.2024.3465564.
- [48] Qiu R, Yin Y, Su Q, Guan T. Func-Bagging: An Ensemble Learning Strategy for Improving the Performance of Heterogeneous Anomaly Detection Models. *Applied Sciences* 2025;15(2):905. <https://doi.org/10.3390/app15020905>.
- [49] J. Zhang, H. Sui, X. Sun, C. Ge, L. Zhou and W. Susilo, "GrabPhisher: Phishing Scams Detection in Ethereum via Temporally Evolving GNNs," in *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 3727-3741, Nov.-Dec. 2024, doi: 10.1109/TSC.2024.3411449.
- [50] Gao H, Xin R, Chen P, et al. Memory-augment graph transformer based unsupervised detection model for identifying performance anomalies in highly-dynamic cloud environments. *J Cloud Comp* 2025;14:40. <https://doi.org/10.1186/s13677-025-00766-5>.
- [51] Song Y, Zhang D, Wang J, et al. Application of deep learning in malware detection: a review. *J Big Data* 2025;12:99. <https://doi.org/10.1186/s40537-025-01157-y>.
- [52] Meng Yue, Huayang Yan, Ruize Han, Zhijun Wu, "A DDoS attack detection method based on IQR and DFFCNN in SDN", *Journal of Network and Computer Applications*, Volume 240, 2025, 104203, ISSN 1084-8045, Doi: 10.1016/j.jnca.2025.104203.
- [53] Wei Zheng, Chang Liu, Peiran Deng, Xiang Chen, Xiaoxue Wu, "Enhancing concurrency vulnerability detection through AST-based static fuzz mutation", *Journal of Systems and Software*, Volume 222, 2025, 112352, ISSN 0164-1212, Doi: 10.1016/j.jss.2025.112352.
- [54] Zengqiang X, Qiangqiang L, Youfeng L, et al. NDLS: A New Deep Learning-based Approach to Smart Contract Vulnerability Detection. *J Sign Process Syst* 2025;97: 49–68. <https://doi.org/10.1007/s11265-025-01954-x>.
- [55] Alazab A, et al. Enhancing Privacy-Preserving Intrusion Detection through Federated Learning. *Electronics* 2023;12(16):3382. <https://doi.org/10.3390/electronics12163382>.
- [56] Ibrahim Khalaf O, Algburi S, S. A., Selvaraj, D., Sharif, M. S., & Elmedany, W. Federated learning with hybrid differential privacy for secure and reliable cross-IoT platform knowledge sharing. *Security and Privacy* 2024;e374.
- [57] S. Dhanasekaran, P. Gomathi, A. R. Maximus, T. Krishnan and B. Kannan, "Solar Tree based Smart City Street Light Control System using IoT BLYNK Platform," 2022 International Conference on Automation, Computing and Renewable Systems (ICACRS), Pudukkottai, India, 2022, pp. 284-290, doi: 10.1109/ICACRS55517.2022.10029107.
- [58] Lakshminarayanan, R., Dhanasekaran, S., Vinod Kumar, R., & Selvaraj, A. Optimizing federated learning approaches with hybrid Convolutional Neural Networks-Bidirectional Encoder Representations from Transformers for precise estimation of average localization errors in wireless sensor networks. *International Journal of Communication Systems*, e5822.
- [59] UmaRani C, Ramalingam S, Dhanasekaran S, Baskaran K. An hybrid machine learning and improved social spider optimization based clustering and routing protocol for wireless sensor network. *Wireless Networks* 2024;1–26.
- [60] Hamdi N, et al. Federated learning-based intrusion detection system for Internet of Things. *IJIS* 2023;22:1937–48. <https://doi.org/10.1007/s10207-023-00727-6>.
- [61] Chaurasia N, et al. A federated learning approach to network intrusion detection using residual networks in industrial IoT networks. *JSC* 2024;80:18325–46. <https://doi.org/10.1007/s11227-024-06153-2>.
- [62] Bhavsar MH, et al. FL-IDS: Federated Learning-Based Intrusion Detection System Using Edge Devices for Transportation IoT. *IEEE Access* 2024;12:52215–26. <https://doi.org/10.1109/ACCESS.2024.3386631>.
- [63] Lazzarini R, et al. Federated Learning for IoT Intrusion Detection. *AI* 2023;4(3): 509–30. <https://doi.org/10.3390/ai4030028>.
- [64] Amandeep Kaur, et al., (2024), "Intrusion Detection Approach for Industrial Internet of Things Traffic using Deep Recurrent Reinforcement Learning Assisted Federated Learning", in *IEEE TAI*, DOI: 10.1109/TAI.2024.3443787.
- [65] <https://www.kaggle.com/datasets/subhajournal/iotintrusion> accessed on 16th May 2024.